

09

CAPÍTULO

Ambiente Profissional de Desenvolvimento

Como equipes de engenharia desenvolvem soluções embarcadas modernas: DevOps, CI/CD, Git, OTA, testes automatizados e fluxo profissional completo.

Este capítulo apresenta o ambiente profissional completo de desenvolvimento para projetos UNIIKER, do primeiro commit a operação em produção. DevOps aplicado a sistemas embarcados, organização de projetos, controle de versão com Git Flow, CI/CD pipelines, OTA com rollback, testes automatizados, observabilidade e segurança do desenvolvimento. Seis novas caixas editoriais orientam a configuração de um fluxo de engenharia moderno.

DEVOPS

CI/CD

GIT FLOW

OTA

AUTOMAÇÃO

QUALIDADE

Sumário do Capítulo

9.1 Engenharia de Desenvolvimento Moderno	3
9.1.1 Evolução do Desenvolvimento Embarcado	3
9.1.2 Cultura DevOps Aplicada a Embarcados	4
9.2 Ambiente de Desenvolvimento	5
9.2.1 Editores e IDEs	5
9.2.2 Ferramentas de Build e Deploy	6
9.2.3 Ferramentas de Debug	6
9.3 Organização de Projetos	6
9.3.1 Estrutura de Diretórios	7
9.3.2 Convenções de Nomenclatura	8
9.4 Controle de Versão	8
9.4.1 Git Flow vs Trunk-Based Development	9
9.4.2 Versionamento Semântico	10
9.5 Integração Contínua	10
9.5.1 Estágios do Pipeline	11
9.6 OTA (Over-The-Air Update)	12
9.6.1 Arquitetura OTA com Rollback	13
9.7 Logs e Observabilidade	13
9.7.1 Três Pilares da Observabilidade	13
9.8 Testes Automatizados	14
9.8.1 Hardware-in-the-Loop (HIL)	14
9.9 Documentação Técnica	15
9.10 Segurança do Desenvolvimento	15
9.10.1 Credenciais, Chaves e Certificados	15
9.10.2 Assinatura de Firmware e Cadeia de Confiança	16
9.11 Fluxo Profissional de Desenvolvimento	16
9.11.1 As 11 Etapas	17
9.12 Estudos de Caso	18

9.12.1 Smart City	18
9.12.2 ETA Inteligente	18
9.12.3 Agricultura Inteligente	18
9.12.4 Robótica	18
9.12.5 Educação	19
9.12.6 Laboratório Universitário	19
9.13 Conclusão	19
9.13.1 Amador vs Profissional	19
9.13.2 Reduzindo Riscos e Aumentando Qualidade	19
Referências Bibliográficas	20

CAPÍTULO 9

Ambiente Profissional de Desenvolvimento

DevOps, CI/CD, Git, OTA e fluxo profissional para projetos UNIHAKER

Este capítulo inicia a Parte III do livro, dedicada a engenharia profissional. Após dominar hardware (Capítulos 3-4), escolha de plataforma (Capítulo 5), arquitetura (Capítulo 6), catálogo de projetos (Capítulo 7) e metodologia UEF (Capítulo 8), o leitor agora aprende como equipes profissionais de engenharia organizam, automatizam e conduzem o desenvolvimento de soluções embarcadas inteligentes com UNIHAKER. O foco não está em ferramentas isoladas, mas em como todas elas se integram em um fluxo de engenharia moderno, do primeiro commit a operação em produção.

9.1 Engenharia de Desenvolvimento Moderno

O desenvolvimento de sistemas embarcados evoluiu dramaticamente nas últimas décadas. O que era, nos anos 1990, uma atividade essencialmente individual - um engenheiro escrevendo firmware em C sobre um microcontrolador 8051, sem versionamento, sem testes automatizados, sem CI/CD - tornou-se, nos anos 2020, uma disciplina colaborativa que integra hardware, firmware, software cloud, modelos de IA e infraestrutura DevOps. Esta seção analisa essa evolução e introduz a cultura DevOps aplicada a sistemas embarcados.

9.1.1 Evolução do Desenvolvimento Embarcado

O desenvolvimento embarcado tradicional seguia o modelo cascata: requisitos, design, implementação, teste, deploy. Cada fase era sequencial e estanque. Firmware era escrito em C assembly, compilado localmente, gravado em flash via JTAG, e testado manualmente com osciloscópio. Versionamento, quando existia, era por cópias de diretórios com sufixos de data. Equipes eram pequenas (1-3 engenheiros) e comunicação era face a face. Esse modelo funcionou por décadas porque sistemas embarcados eram relativamente simples: poucos milhares de linhas de código, um microcontrolador, sem conectividade, sem IA, sem cloud.

O desenvolvimento embarcado moderno é radicalmente diferente. Sistemas UNIHAKER combinam firmware (MicroPython ou C/C++), software cloud (Python, Node.js, Docker), modelos de IA (TensorFlow Lite, MediaPipe), dashboards (Grafana, Node-RED) e infraestrutura DevOps (CI/CD, OTA, monitoramento). Equipes são multidisciplinares: engenheiros de firmware, de hardware, de cloud, de IA, de DevOps, de QA. O código total de um projeto profissional facilmente excede 50.000 linhas, distribuídas em dezenas de repositórios. Versionamento é obrigatório. Testes automatizados são essenciais. CI/CD é padrão. OTA é requisito. Sem essas práticas, projetos modernos são inadmissíveis.

▲ INDICADOR DE MATURIDADE

Nível de Maturidade DevOps:

Nível 1 (Inicial): desenvolvimento manual, sem Git, sem CI.

Nível 2 (Gerenciado): Git + build manual.

Nível 3 (Definido): CI com testes automáticos.

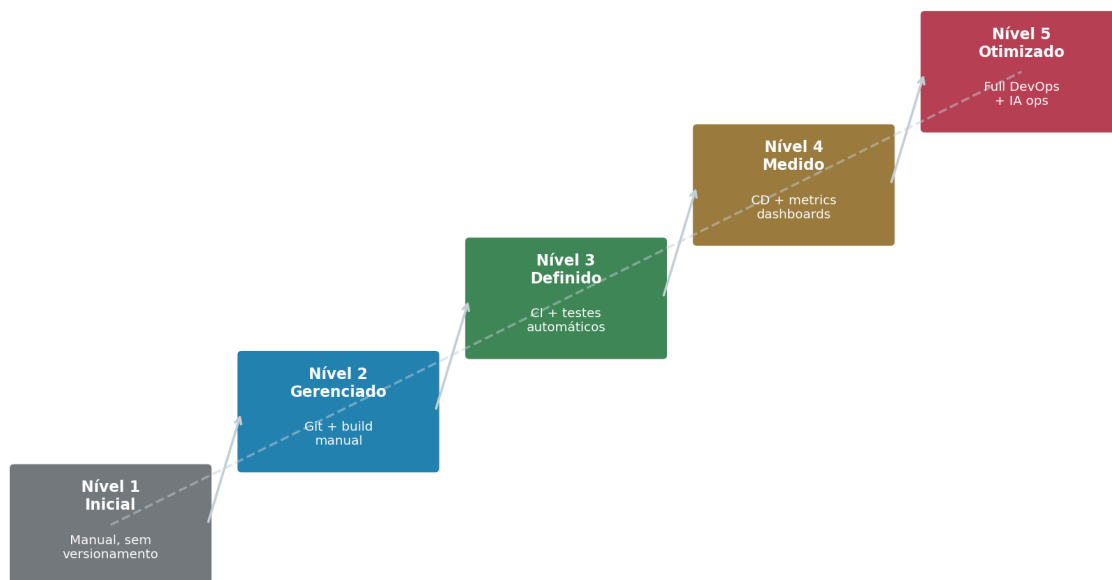
Nível 4 (Medido): CD + métricas + dashboards.

Nível 5 (Otimizado): full DevOps + IA ops.

Maioria dos projetos UNIHAKER em 2025: Nível 2-3.

Meta para equipes profissionais: Nível 4.

Modelo de Maturidade DevOps para Sistemas Embarcados



Cada nível requer investimento em ferramentas, processos e cultura

Figura 9.1 - Modelo de maturidade DevOps para sistemas embarcados, em 5 níveis. Cada nível requer investimento em ferramentas, processos e cultura. Fonte: elaborado pelo autor.

9.1.2 Cultura DevOps Aplicada a Embarcados

DevOps não é apenas ferramentas - é cultura. Os três princípios fundamentais do DevOps aplicados a sistemas embarcados são: (i) **automação** - tudo que é repetitivo deve ser automatizado (build, teste, deploy, calibração); (ii) **colaboração** - desenvolvedores (Dev) e operadores (Ops) trabalham juntos, não em silos; (iii) **feedback contínuo** - métricas de produção alimentam melhorias no desenvolvimento. No contexto UNIHAKER, DevOps significa: firmware desenvolvido com Git Flow, build automatizado via CI, testes HIL (Hardware-in-the-Loop) no pipeline, deploy via OTA com rollback automático, e monitoramento remoto via Grafana que informa a próxima iteração de desenvolvimento.

▲ VISÃO DO LIDER TECNICO

Visão do Lider Técnico: a pergunta que todo Tech Lead deveria fazer ao avaliar um projeto embarcado não é «quantas linhas de código temos?» mas «quantas horas por semana a equipe gasta em tarefas manuais repetitivas?». Se a resposta for mais de 10 horas, ha oportunidade de automação. As 10 horas investidas em automatizar um build se pagam em 5 semanas. As 20 horas investidas em CI/CD se pagam em 2 meses. DevOps não é custo -é investimento com ROI mensurável.

9.2 Ambiente de Desenvolvimento

O ambiente de desenvolvimento é o conjunto de ferramentas que engenheiros utilizam para escrever, testar, depurar e deployar código. Esta seção apresenta e compara as principais ferramentas relevantes para projetos UNIHAKER, explicando quando utilizar cada uma.

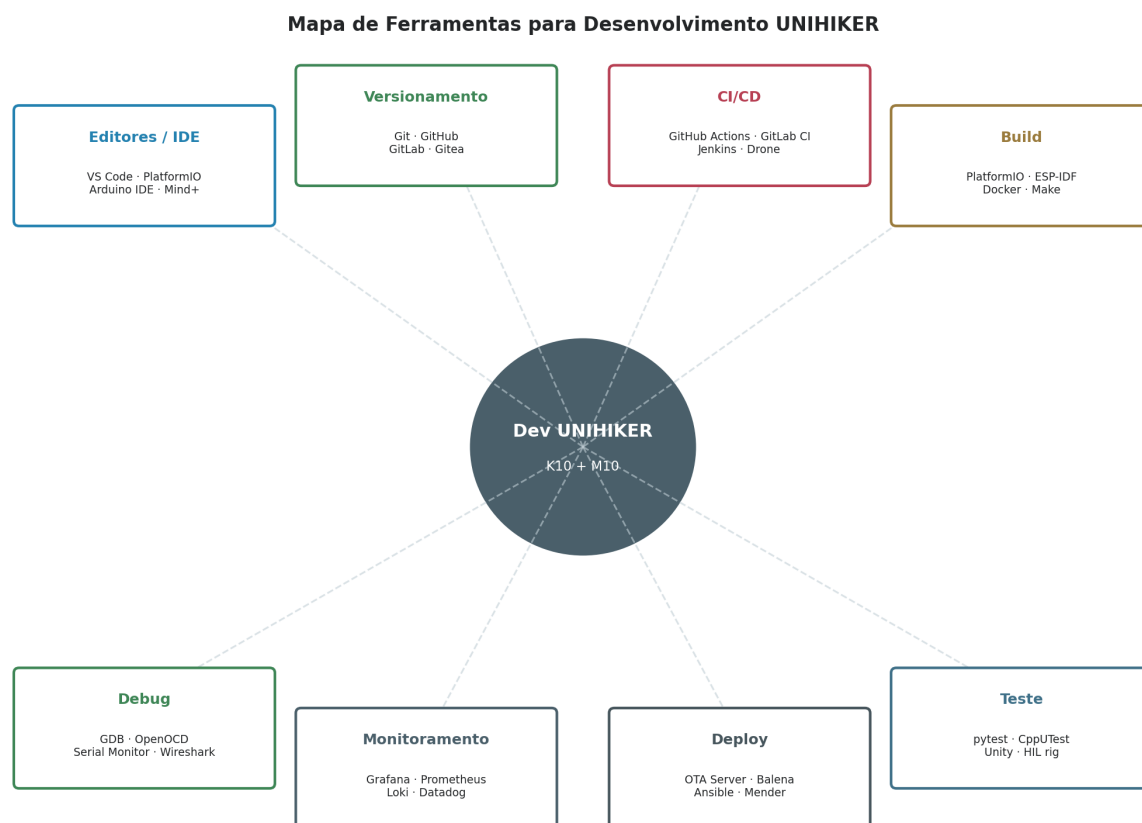


Figura 9.2 - Mapa de ferramentas para desenvolvimento UNIHAKER, organizado em 8 categorias: editores, versionamento, CI/CD, build, teste, deploy, monitoramento e debug. Fonte: elaborado pelo autor.

9.2.1 Editores e IDEs

VS Code é o editor de fato para desenvolvimento profissional em Python, C/C++ e JavaScript. Leve, extensível, com milhares de extensões, integração nativa com Git, terminal embutido, depuração. Para UNIHAKER, extensões essenciais: Python, PlatformIO, Docker, GitLens, Remote SSH (para desenvolver no

M10 remotamente). **PlatformIO** é IDE especializada em sistemas embarcados, integrada ao VS Code. Gerencia toolchains, bibliotecas, placas, build e upload. Suporta ESP32-S3 (K10) e é a ferramenta recomendada para firmware C/C++. **Arduino IDE** é mais simples, adequado para iniciantes ou prototipagem rápida, mas limitado para projetos profissionais (sem debug avançado, sem CI nativo). **Mind+** é IDE gráfico da DFRobot com blocos visuais para iniciantes, gera código Python automaticamente.

▲ FERRAMENTA RECOMENDADA

Cenário: Equipe profissional desenvolvendo firmware para K10 e M10.

Ferramenta recomendada: VS Code + PlatformIO

Por que: VS Code oferece editor universal com debug, Git e terminal. PlatformIO gerencia toolchain ESP-IDF, bibliotecas e build de forma reproduzível. Integração via extensão PlatformIO no VS Code combina o melhor dos dois. Para M10 (Linux), usar VS Code Remote SSH para editar diretamente no M10.

Alternativas: Arduino IDE (simples, limitado), Mind+ (blocos, para iniciantes), Eclipse (pesado, pouco usado em 2025).

9.2.2 Ferramentas de Build e Deploy

PlatformIO é a ferramenta de build recomendada para firmware do K10 (ESP32-S3). Gerencia dependências, configura toolchain, compila e faz upload via USB. Arquivo de configuração *platformio.ini* define placa, framework (Arduino ou ESP-IDF), bibliotecas e opções de build. **ESP-IDF** (Espressif IoT Development Framework) é o framework nativo da Espressif, mais potente que Arduino Core mas mais complexo. Recomendado para projetos avançados que precisam de controle total do hardware. **Docker** é essencial para o M10: encapsula aplicações Python, Node-RED, Grafana, InfluxDB em containers reproduzíveis. **Make/CMake** podem ser usados para builds complexos multi-plataforma.

9.2.3 Ferramentas de Debug

GDB (GNU Debugger) é o depurador padrão para C/C++. Para ESP32-S3, integrado com **OpenOCD** (Open On-Chip Debugger) via interface JTAG. Permite breakpoints, watch variables, stack trace, memory inspection. Para MicroPython (K10), debug é feito via REPL serial ou VS Code Python debugger. Para M10 (Linux), debug nativo com pdb/gdb sobre SSH. **Serial Monitor** (PlatformIO ou screen/minicom) é essencial para ver logs em tempo real durante desenvolvimento. **Wireshark** analisa tráfego de rede (MQTT, HTTP, BLE) para depurar comunicação.

9.3 Organização de Projetos

A organização de um projeto de software é fundamental para sua manutenibilidade. Projetos desorganizados - com arquivos espalhados, nomes inconsistentes, documentação ausente - tornam-se inadministráveis em meses. Esta seção apresenta metodologia estruturada para organização de projetos UNIHIKER.

Estrutura de Diretórios Recomendada para Projeto UNIHAKER

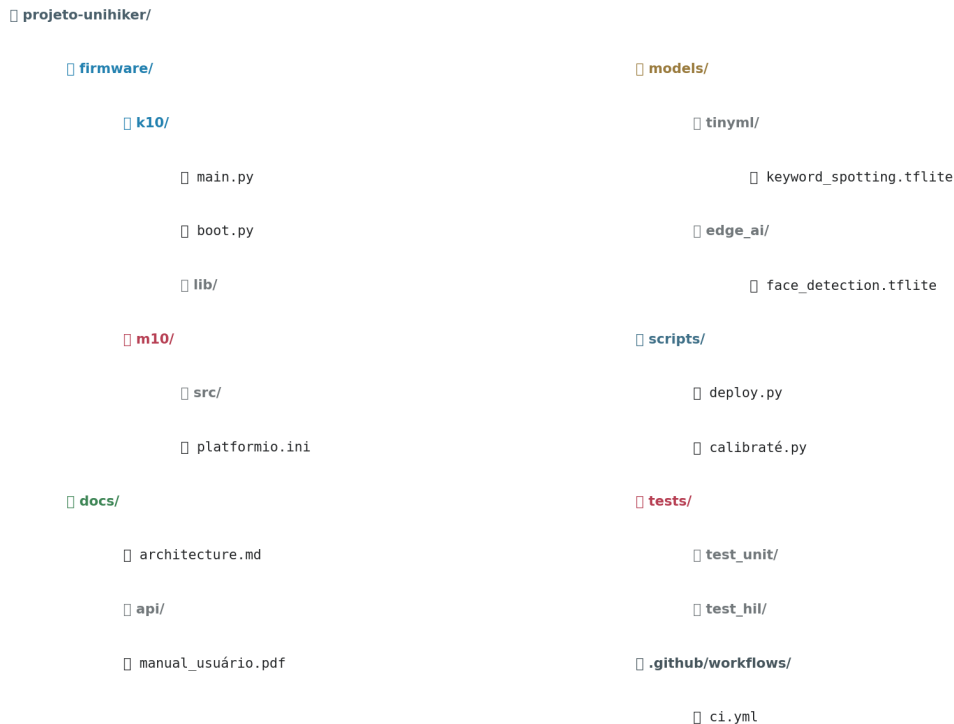


Figura 9.3 - Estrutura de diretórios recomendada para projeto UNIHAKER, separando firmware, documentação, modelos de IA, scripts, testes e CI/CD. Fonte: elaborado pelo autor.

9.3.1 Estrutura de Diretórios

A estrutura recomendada separa claramente componentes por tipo: **firmware/** contém código do K10 (MicroPython) e M10 (C/C++ via PlatformIO); **docs/** contém documentação de arquitetura, API e manuais; **models/** contém modelos de IA (TinyML e Edge AI) separados por plataforma; **scripts/** contém scripts de automação (deploy, calibração, teste); **tests/** contém testes unitários e HIL; **.github/workflows/** contém pipelines de CI/CD. Essa separação permite que membros diferentes da equipe trabalhem em paralelo sem conflitos, e facilita onboarding de novos membros.

▲ ESTRUTURA RECOMENDADA

Estrutura recomendada para projeto UNIHIKER:

```
projeto-unihiker/  
firmware/  
k10/ (MicroPython: main.py, boot.py, lib/)  
m10/ (C/C++: src/, platformio.ini)  
docs/ (architecture.md, api/, manual_usuario.pdf)  
models/  
tinymml/ (modelos .tflite para K10)  
edge_ai/ (modelos .tflite para M10)  
scripts/ (deploy.py, calibrate.py, test_hil.py)  
tests/  
test_unit/ (pytest, CppUTest)  
test_hil/ (Hardware-in-the-Loop)  
.github/workflows/ (ci.yml, deploy.yml)  
docker/ (Dockerfiles para M10)  
README.md  
CHANGELOG.md  
LICENSE  
.gitignore  
.gitattributes
```

9.3.2 Convenções de Nomenclatura

Convenções de nomenclatura consistentes são essenciais para legibilidade e manutenibilidade. Recomendações: arquivos Python em *snake_case.py*; arquivos C/C++ em *snake_case.c* ou *PascalCase.cpp*; classes em *PascalCase*; funções e variáveis em *snake_case*; constantes em *UPPER_SNAKE_CASE*; branches Git em *feature/descrição*, *fix/descrição*, *release/vX.Y.Z*; tags Git em *vX.Y.Z* (versionamento semântico). Documentação em Markdown. Configuração em YAML ou TOML (não JSON, que não suporta comentários).

9.4 Controle de Versão

Controle de versão é a fundação de qualquer projeto profissional de software. Git é o padrão de facto da indústria, com plataformas como GitHub e GitLab oferecendo hospedagem, CI/CD integrado e colaboração. Esta seção analisa profundamente o uso de Git em projetos embarcados.

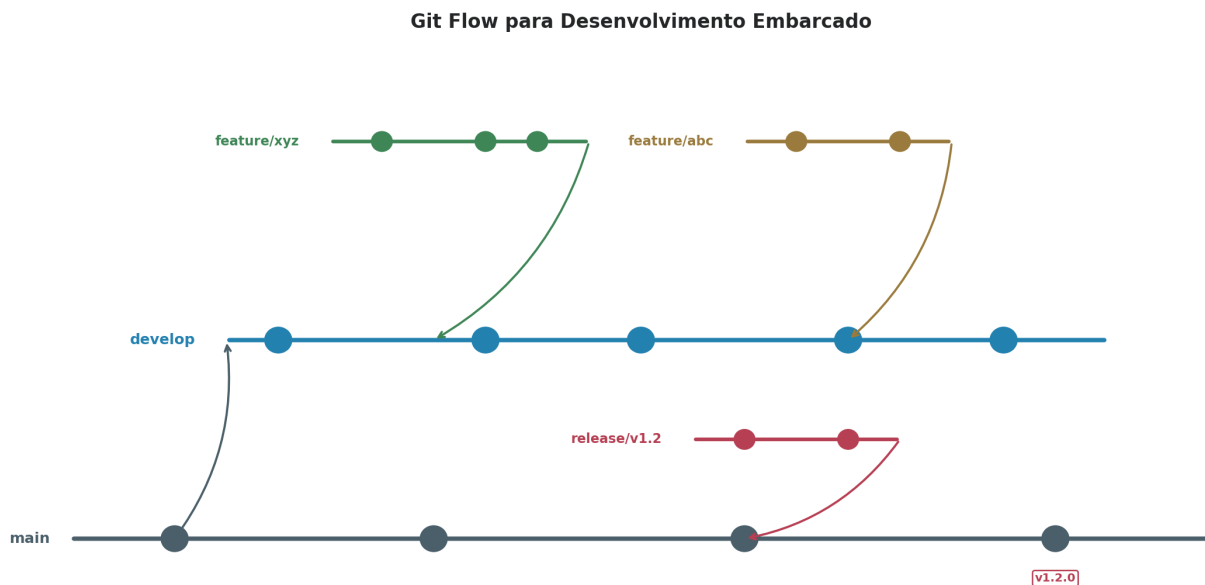


Figura 9.4 - Git Flow para desenvolvimento embarcado: branch *main* (produção), *develop* (integração), *feature/* (desenvolvimento), *release/* (estabilização). Tags marcam versões semânticas. Fonte: elaborado pelo autor.

9.4.1 Git Flow vs Trunk-Based Development

Git Flow é o modelo de branching mais usado em projetos embarcados. Branch *main* contém apenas código em produção, tagged com versões semânticas. Branch *develop* é a linha de integração onde features são mergeadas. Branches *feature/descrição* são criadas a partir de *develop* para cada nova funcionalidade. Branches *release/vX.Y.Z* são criadas para estabilizar versões antes de produção. Branches *hotfix/descrição* são criadas a partir de *main* para correções urgentes em produção. Esse modelo oferece rastreabilidade excelente mas pode ser lento para equipes pequenas.

Trunk-Based Development é alternativa mais ágil: todos commitam diretamente em *main* (ou em branches de vida curta, máximo 24h). CI/CD rígido garante que *main* está sempre em condição de deploy. Feature flags ativam novas funcionalidades gradualmente. Esse modelo é mais rápido mas exige disciplina de CI/CD e testes automatizados robustos. Para projetos UNIHAKER, Git Flow é recomendado para equipes de 3+ pessoas; trunk-based para equipes menores ou startups em fase de iteração rápida.

❓ TRADE-OFF DE ENGENHARIA

Trade-off: Git Flow vs Trunk-Based

Git Flow: mais estrutura, melhor rastreabilidade, mais lento. Ideal para firmware embarcado com releases periódicos (mensais/trimestrais).

Trunk-Based: mais ágil, menos burocracia, requer CI/CD maduro. Ideal para software cloud com deploy contínuo.

Para UNIHAKER: Git Flow para firmware (K10), trunk-based para software cloud (M10 apps). Híbrido e válido - adaptar a realidade do projeto.

9.4.2 Versionamento Semântico

Versionamento semântico (SemVer) usa formato **MAJOR.MINOR.PATCH** (ex.: v2.3.1). **MAJOR** incrementa quando ha mudancas incompativeis (breaking changes). **MINOR** incrementa quando ha novas funcionalidades compatíveis. **PATCH** incrementa para correções de bugs compatíveis. Para firmware embarcado, SemVer e critico porque OTA depende dele: dispositivo verifica versão atual vs versão disponível e decide se atualiza. Mudancas MAJOR podem exigir intervenção manual; MINOR e PATCH são atualizaveis automaticamente. Tags Git (*git tag v2.3.1*) marcam releases e são usadas por CI/CD para gerar artefatos de OTA.

9.5 Integração Contínua

Integração Contínua (CI) e a prática de integrar código frequentemente (varias vezes ao dia) em branch compartilhado, com build e testes automatizados a cada integração. Continuous Delivery (CD) estende CI com deploy automático para ambientes de staging e produção. Esta seção analisa como implementar CI/CD em projetos UNIHAKER.

Arquitectura de Pipeline CI/CD para UNIHIER

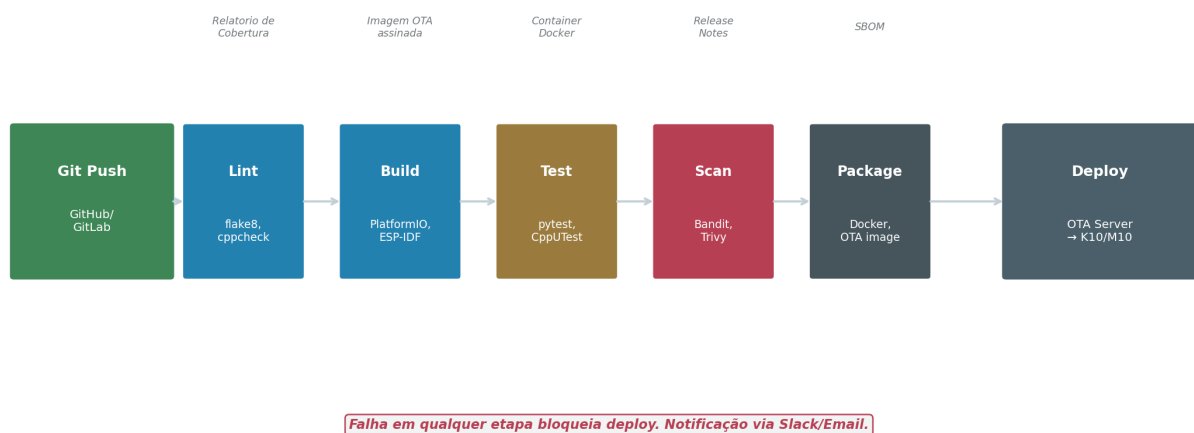


Figura 9.5 - Arquitetura de pipeline CI/CD para UNIHAKER: Git Push dispara Lint, Build, Test, Scan, Package, Deploy. Falha em qualquer etapa bloqueia deploy. Fonte: elaborado pelo autor.



Figura 9.6 - Pipeline DevOps completo: Commit, Build, Test, Package, Deploy Dev, Teste Campo, Deploy Prod, Monitor. Feedback loop conecta monitoramento a commit. Fonte: elaborado pelo autor.

9.5.1 Estágios do Pipeline

Um pipeline CI/CD profissional para UNIHAKER compreende cinco estágios principais. **Lint**: análise estática de código (flake8 para Python, cppcheck para C/C++, bandit para segurança). Bloqueia commits com erros de estilo. **Build**: compilação de firmware (PlatformIO para K10, GCC para M10) e build de containers Docker para M10. **Test**: execucao de testes unitarios (pytest, CppUTest), testes de integração e cobertura de código (coverage.py). **Scan**: análise de vulnerabilidades (Trivy para Docker, safety para Python deps). **Package**: geração de artefatos (firmware .bin assinado, imagem Docker, SBOM).

▲ PIPELINE DE ENGENHARIA

Pipeline de Engenharia recomendado para UNIHAKER:

Trigger: push para develop ou PR para main

1. Lint (flake8 + cppcheck + bandit) - 30s
2. Build K10 (PlatformIO compile) - 2min
3. Build M10 (Docker build) - 3min
4. Unit tests (pytest + CppUTest) - 1min
5. Coverage report (codecov) - 30s
6. Security scan (Trivy + safety) - 1min
7. Package firmware + sign + upload - 1min
8. Deploy to staging (OTA canary 10%) - 30s
9. HIL tests on staging - 5min
10. Promote to production (manual approval) - manual

Total automatizado: ~15 min

Ferramentas: GitHub Actions ou GitLab CI

9.6 OTA (Over-The-Air Update)

OTA é a capacidade de atualizar firmware remotamente, sem conexão física. Para projetos com 100+ dispositivos, OTA é essencial - atualizar fisicamente cada dispositivo é inviável. Esta seção analisa OTA em profundidade.

Fluxo Completo de OTA com Rollback Automático



Figura 9.7 - Fluxo completo de OTA com rollback automático: Build CI gera firmware assinado, OTA server armazena, dispositivo baixa e verifica, instala em particao backup, testa boot, confirma ou reverte. Fonte: elaborado pelo autor.

9.6.1 Arquitetura OTA com Rollback

OTA seguro exige arquitetura de particao dupla (dual-bank). O K10 (ESP32-S3) suporta nativamente duas particoes OTA no flash. O dispositivo inicializa da particao ativa, baixa nova versao para particao inativa, verifica assinatura digital, reinicializa testando a nova versao. Se o watchdog não for alimentado em tempo definido (tipicamente 60 segundos), o bootloader reverte automaticamente para particao anterior. Esse mecanismo garante que firmware corrompido nunca deixa dispositivo inoperante. Para o M10 (Linux), rollback é implementado via A/B partitions com systemd-boot ou U-Boot.

▲ SEGURANÇA EM FOCO

Segurança OTA - Cadeia de Confiança:

1. **Chave privada** guardada em HSM ou cofre, nunca no repositório
2. **CI/CD assina firmware** com chave privada após build
3. **Firmware assinado** e publicado no OTA server
4. **Dispositivo verifica assinatura** com chave publica embutida no bootloader
5. **Somente firmware assinado** e instalado - atacante não pode injetar firmware malicioso
6. **Rollback automático** protege contra firmware instável

Sem assinatura digital, OTA é vetor de ataque. Nunca deployar OTA sem assinatura em produção.

9.7 Logs e Observabilidade

Observabilidade é a capacidade de entender o estado interno de um sistema observando suas saidas externas. Em sistemas embarcados distribuídos, observabilidade é crítica: sem logs, métricas e tracing, é impossível diagnosticar problemas em dispositivos remotos. Esta seção analisa os três pilares da observabilidade.

9.7.1 Três Pilares da Observabilidade

Logs são eventos discretos com timestamp: «sensor DHT22 leu 24.3C em 2025-02-15T10:30:00Z». Devem ser estruturados (JSON, não texto livre), centralizados (enviados ao M10 ou cloud), com níveis (DEBUG, INFO, WARNING, ERROR). Ferramentas: Loki + Grafana para self-hosted, CloudWatch/Stackdriver para cloud. **Métricas** são valores numericos ao longo do tempo: CPU usage, RAM, temperatura, latência MQTT. Coletadas via Prometheus no M10, visualizadas em Grafana. **Tracing** rastreia requisicoes atraves de componentes: K10 -> MQTT -> M10 -> Cloud. Essencial para diagnosticar latência em sistemas distribuídos. Ferramenta: Jáeger ou Zipkin.

▲ INDICADOR DE MATURIDADE

Indicador de Maturidade de Observabilidade:

Nível 1: logs seriais (printf/debug_print)
 Nível 2: logs estruturados locais (JSON no M10)
 Nível 3: logs centralizados (Loki + Grafana)
 Nível 4: + métricas (Prometheus) + alertas
 Nível 5: + tracing distribuído (Jáeger) + AIOps

Meta para produção: Nível 4 mínimo.

Sem observabilidade adequada, operar 100+ dispositivos e como dirigir de olhos vendados.

9.8 Testes Automatizados

Testes automatizados são a única defesa sistemática contra regressões. Em projetos embarcados, testes são particularmente desafiadores porque envolvem hardware real. Esta seção apresenta metodologia de testes em cinco níveis.

Nível	O que Testa	Ferramenta	Frequência
Unitario	Funções individuais	pytest, CppUTest	A cada commit
Integração	Modulos juntos	pytest + mocks	A cada PR
HIL	Hardware real	Rig automatizado	Diario (nightly)
Regressão	Versão vs versão	pytest suite completa	Pre-release
Estrêsse	Carga, duração	Script 24-72h	Semanal
Desempenho	Latência, throughput	Benchmark suite	Pre-release

Tabela 9.1 - Seis níveis de testes automatizados para projetos UNIHAKER. Fonte: elaborado pelo autor.

9.8.1 Hardware-in-the-Loop (HIL)

HIL é o nível mais valioso é mais complexo de teste embarcado. Um **rig HIL** é um banco de testes físico com: K10/M10 montado, sensores e atuadores conectados, fonte de alimentação controlada, instrumentação (osciloscópio, multímetro) via SCPI, e computador de controle que executa cenários de teste automatizados. O rig simula eventos reais: aplica temperatura variada (câmara térmica), injeta sinais de sensor, corta alimentação, mede resposta do dispositivo. Cenários típicos: «dispositivo sobrevive a corte de alimentação por 100ms», «sensor de temperatura lê 25C com +/- 0.5C de precisão após 1h de operação», «OTA com rollback funciona em 100% dos testes».

▲ VISÃO DO LIDER TECNICO

Visão do Líder Técnico sobre testes: a maior resistência da equipe a testes automatizados e «não temos tempo para escrever testes». Resposta: vocês já têm tempo para debugar problemas em produção? Tempo investido em testes e tempo economizado em debug. Equipes maduras gastam 30-40% do tempo em testes. Equipes imaturas gastam 60%+ do tempo em debug de produção. A escolha é clara. Comece com testes unitários (mais fáceis), evolua para HIL (mais complexo). Não tente fazer tudo de uma vez.

9.9 Documentação Técnica

Documentação é frequentemente a parte mais negligenciada do desenvolvimento e a primeira a sofrer com prazos apertados. Ironia: é também a primeira coisa que novos membros da equipe precisam. Esta seção analisa os sete tipos de documentação essenciais.

Documento	Público	Conteúdo	Ferramenta
README.md	Todos	Visão geral, setup, build	Markdown
Arquitetura	Engenheiros	Diagramas, decisões	Markdown + draw.io
API	Desenvolvedores	Endpoints, schemas	OpenAPI/Swagger
Changelog	Todos	Mudanças por versão	Markdown
Manual usuário	Usuários	Como usar, FAQ	PDF/Markdown
Manual técnico	Manutenção	Troubleshooting, hardware	PDF
Runbook ops	On-call	Procedimentos incidente	Markdown

Tabela 9.2 - Sete tipos de documentação técnica essencial. Fonte: elaborado pelo autor.

9.10 Segurança do Desenvolvimento

Segurança do desenvolvimento é diferente de segurança do produto (tratada no Capítulo 6). Trata-se de proteger o processo de desenvolvimento: credenciais, chaves, código-fonte, pipeline CI/CD. Esta seção analisa as seis dimensões críticas.

9.10.1 Credenciais, Chaves e Certificados

Nunca commitar credenciais (senhas, chaves API, certificados) em repositório Git. Usar **variáveis de ambiente** ou **secret managers** (GitHub Secrets, GitLab CI Variables, AWS Secrets Manager, HashiCorp Vault). Arquivo `.gitignore` deve bloquear arquivos de credencial (`.env`, `*.key`, `*.pem`). Ferramentas como **git-secrets** ou **truffleHog** escaneiam commits procurando credenciais vazadas. Se uma credencial for commitada por engano, deve ser **revogada imediatamente** - não basta remover do Git, pois histórico permanece.

▲ SEGURANÇA EM FOCO

Segurança em Foco - SBOM (Software Bill of Materials):

SBOM é inventário completo de todas as dependências de software em um produto, incluindo versões e licenças. Exigido por regulamentações recentes (EU Cyber Resilience Act, US Executive Order 14028). Para UNIHIKER, SBOM inclui: bibliotecas Python (requirements.txt), pacotes Debian (dpkg --get-libs), containers Docker (docker scan), bibliotecas PlatformIO. Ferramentas: Syft (gera SBOM), Gype (analisa vulnerabilidades), Dependency-Track (monitora continuamente). **Sem SBOM, você não sabe o que está rodando em produção.**

9.10.2 Assinatura de Firmware e Cadeia de Confiança

Cadeia de confiança (chain of trust) é sequência de verificações criptográficas que garantem autenticidade e integridade em cada estágio do ciclo de vida do firmware. Começa com **secure boot**: bootloader verifica assinatura do firmware antes de executá-lo. Firmware é assinado no CI/CD com chave privada. Chave pública correspondente está embutida no bootloader. Apenas firmware assinado pela chave privada correta é executado. Para OTA, dispositivo verifica assinatura do novo firmware antes de instalar. Se verificação falhar, firmware é rejeitado e versão atual mantida. Essa cadeia protege contra ataques de injeção de firmware malicioso.

9.11 Fluxo Profissional de Desenvolvimento

Esta seção consolida todos os conceitos anteriores em um fluxograma completo de desenvolvimento profissional, da ideia à melhoria contínua.

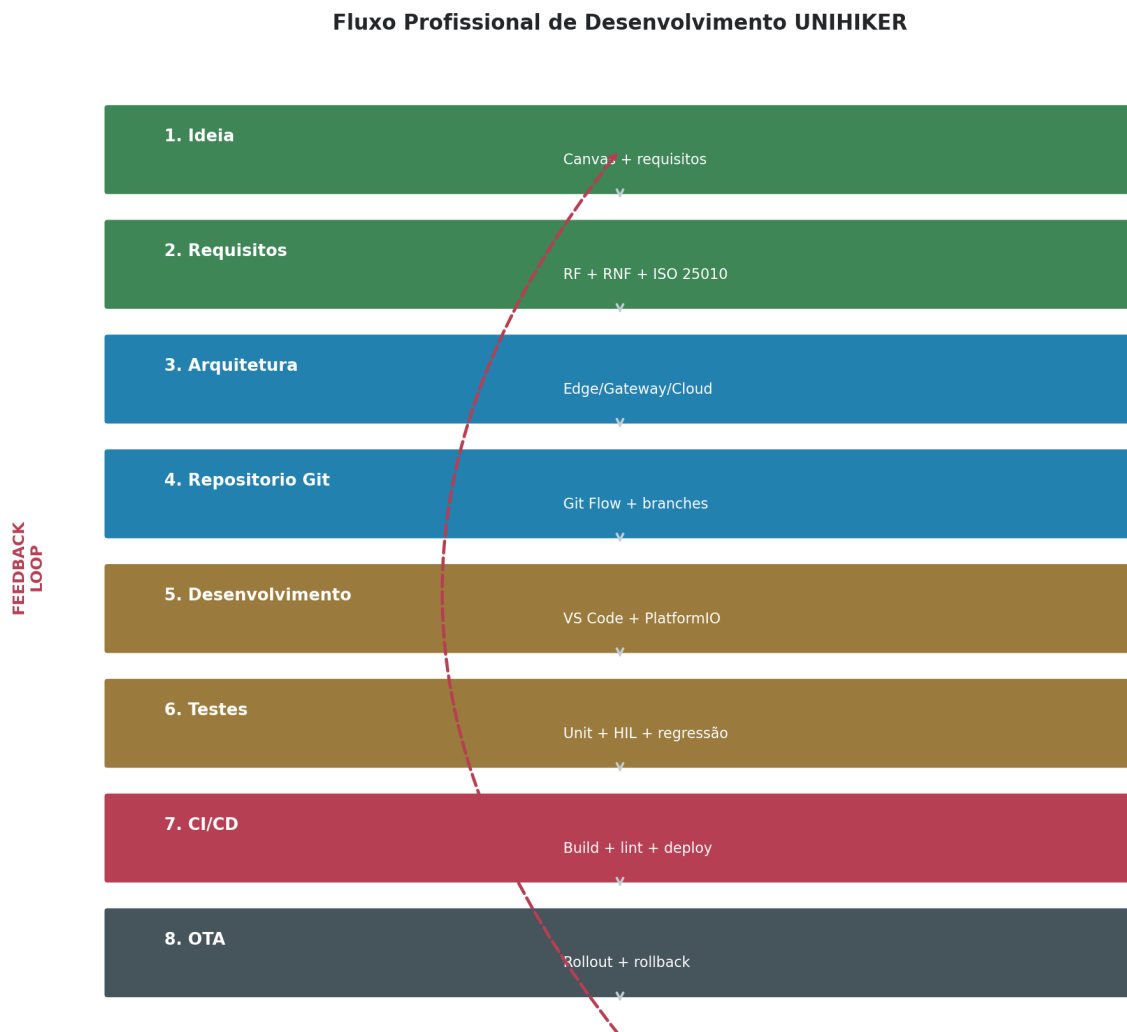


Figura 9.8 - Fluxo profissional de desenvolvimento UNIHIKER em 11 etapas, da Ideia a Melhoria Contínua, com feedback loop.
 Fonte: elaborado pelo autor.

9.11.1 As 11 Etapas

- 1. Ideia:** Canvas do projeto, identificação do problema, stakeholders alinhados.
- 2. Requisitos:** Requisitos funcionais e não funcionais (ISO 25010), restrições, premissas, critérios de aceitação.
- 3. Arquitetura:** Diagrama arquitetural Edge/Gateway/Cloud, escolha K10/M10, protocolos, IA.
- 4. Repositório Git:** Git Flow, branches, .gitignore, README inicial, estrutura de diretórios.
- 5. Desenvolvimento:** VS Code + PlatformIO, feature branches, commits atômicos, code review via PR.
- 6. Testes:** Unitários, integração, HIL. Cobertura mínima 70%. Rodados em CI a cada commit.
- 7. CI/CD:** Pipeline automatizado: lint, build, test, scan, package, deploy staging. Promote to prod com approval manual.
- 8. OTA:** Rollout canário (10%), monitorar 24-48h, expandir para 100%. Rollback automático se falha.

9. Monitoramento: Grafana + Prometheus + Loki. Alertas P1-P4. Dashboard 24/7.

10. Operação: On-call rotation, runbooks, incident response, post-mortem.

11. Melhoria Contínua: Retrospectivas quinzenais, roadmap atualizado, tech debt tracking, métricas de qualidade (MTTR, change failure rate).

▲ VISÃO DO LIDER TECNICO

Visão do Líder Técnico - mensagem final: o fluxo profissional apresentado não é burocracia - e proteção. Cada etapa existe porque equipes que pularam etapas pagaram caro. Projetos sem Git perdem código. Projetos sem CI quebram em produção. Projetos sem OTA não escalam. Projetos sem testes regredem. Projetos sem documentação ficam inmanutíveis. Projetos sem monitoramento são caixa-preta. O fluxo profissional não garante sucesso, mas diminui drasticamente a probabilidade de fracasso. E o que separa amadores de profissionais.

9.12 Estudos de Caso

Esta seção apresenta seis estudos de caso mostrando como equipes profissionais aplicariam o fluxo descrito em domínios distintos.

9.12.1 Smart City

Equipe de 10 engenheiros na prefeitura. Git Flow com main (produção), develop (integração), feature/ por componente (ar, trânsito, iluminação). CI/CD com GitHub Actions: lint Python, build PlatformIO para K10, Docker build para M10, testes pytest, deploy OTA canário. 500 K10 + 50 M10 + Grafana + Prometheus. Monitoramento 24/7 com on-call rotation entre 3 engenheiros. OTA mensal com janela de manutenção domingo 2-4h.

9.12.2 ETA Inteligente

Equipe de 5 engenheiros em companhia de saneamento. Git Flow simplificado (main + develop). CI/CD com GitLab CI. Firmware K10 via PlatformIO, M10 apps em Docker. Testes HIL com rig automatizado (câmera térmica, injeção de sensores). OTA trimestral com aprovação da engenharia civil. Documentação técnica completa para auditoria ANVISA. SBOM gerado automaticamente.

9.12.3 Agricultura Inteligente

Startup com 4 engenheiros. Trunk-based development para iteração rápida. CI/CD com GitHub Actions. K10 solar com LoRaWAN, M10 regional com Node-RED. OTA canário 5% antes de 100%. Monitoramento via Grafana Cloud. Feature flags para experimentar novos algoritmos de irrigação sem deploy completo. Testes HIL com simulador de solo.

9.12.4 Robótica

Equipe universitária de 6 alunos + 1 professor. Git Flow educacional (main + feature/). CI/CD básico: build + lint. Testes unitários com pytest. HIL com robôs reais em laboratório. OTA via USB (não remoto). Documentação em Markdown no GitHub Wiki. Roadmap trimestral baseado em competições.

9.12.5 Educação

Escola com 2 professores + 30 alunos. Repositório Git simplificado (apenas main). Sem CI/CD (projeto educacional). Documentação em README + slides. Testes manuais. Deploy via USB. Mind+ para iniciantes, VS Code para avançados. Foco em aprendizado, não em automação.

9.12.6 Laboratório Universitário

Laboratório de pesquisa com 8 pesquisadores. Git Flow completo. CI/CD com GitLab CI self-hosted. K10 + M10 + Jupyter notebooks no M10. Testes unitários e HIL. OTA para dispositivos de campo. Documentação técnica completa (paper-ready). SBOM para reprodutibilidade científica. Code review obrigatório entre pares.

9.13 Conclusão

9.13.1 Amador vs Profissional

Cinco práticas diferenciam um projeto amador de um projeto profissional. Primeiro: **versionamento Git** com Git Flow estruturado. Amador edita arquivos diretamente; profissional usa branches e code review. Segundo: **CI/CD automatizado**. Amador compila manualmente; profissional tem pipeline que builda, testa e deploya automaticamente. Terceiro: **testes automatizados**. Amador testa manualmente; profissional tem suite de testes que roda a cada commit. Quarto: **OTA com rollback**. Amador atualiza via USB; profissional atualiza remotamente com rollback automático. Quinto: **observabilidade**. Amador não sabe o que acontece em produção; profissional tem dashboards, alertas e logs centralizados.

9.13.2 Reduzindo Riscos e Aumentando Qualidade

Riscos são reduzidos por três mecanismos. **Automação** elimina erro humano: CI/CD garante que build e testes sempre executam, sem depender de memória do desenvolvedor. **Rastreabilidade** permite diagnosticar problemas: Git history mostra quando e por que mudança foi feita; logs centralizados mostram o que aconteceu. **Rollback** protege contra falhas: OTA com rollback automático garante que dispositivo sempre volta a versão funcional se nova versão falhar. Qualidade é aumentada por **testes automatizados** (cobertura medida), **code review** (segundo par de olhos), **análise estática** (lint encontra bugs antes da execução) e **CI discipline** (main sempre verde).

▲ INDICADOR DE MATURIDADE

Indicador de Maturidade Final:

Para avaliar se seu projeto e profissional, responda:

- ✓ Tem Git com Git Flow?
- ✓ Tem CI/CD automatizado?
- ✓ Tem testes com 70%+ cobertura?
- ✓ Tem OTA com rollback?
- ✓ Tem monitoramento 24/7?
- ✓ Tem documentação técnica?
- ✓ Tem SBOM?
- ✓ Tem code review?
- ✓ Tem on-call rotation?

7+ SIM: projeto profissional.

4-6 SIM: projeto em transição.

0-3 SIM: projeto amador (risco alto).

Para o leitor que chegou até aqui, a mensagem final é: o ambiente profissional de desenvolvimento não é luxo de grandes empresas - é necessidade para qualquer projeto que pretenda operar em produção com mais de 10 dispositivos. As ferramentas apresentadas (Git, CI/CD, OTA, testes, observabilidade) são gratuitas ou de baixo custo. O investimento principal é cultural: disciplina para seguir processos mesmo sob pressão de prazo. Equipes que investem em DevOps entregam mais rápido, com menos bugs, e operam com menos stress. Equipes que pulam DevOps entram em crise recorrente de produção. A escolha é sua.

Referências Bibliográficas

As referências a seguir seguem o padrão ABNT NBR 6023:2018.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.

BROOKS, Frederick P. **The mythical man-month: essays on software engineering**. Anniversary ed. Boston: Addison-Wesley, 1995.

DOCKER. **Docker documentation: Container platform**. San Francisco: Docker Inc., 2024. Disponível em: <https://docs.docker.com>. Acesso em: 20 fev. 2025.

ESPRESSIF SYSTEMS. **ESP-IDF programming guide v5.2**. Shanghai: Espressif, 2024. Disponível em: <https://docs.espressif.com/projects/esp-idf/>. Acesso em: 20 fev. 2025.

GITHUB. **GitHub Docs: Actions, Pages e Git Flow**. San Francisco: GitHub, 2024. Disponível em: <https://docs.github.com>. Acesso em: 20 fev. 2025.

GITLAB. **GitLab CI/CD documentation**. San Francisco: GitLab, 2024. Disponível em: <https://docs.gitlab.com/ee/ci/>. Acesso em: 20 fev. 2025.

INCOSE. **INCOSE Systems Engineering Handbook**. 4. ed. Hoboken: Wiley, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 15288: Systems and software engineering - System life cycle processes**. Geneva: ISO, 2015.

- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and software quality requirements and evaluation**. Geneva: ISO, 2011.
- KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. **The DevOps handbook: how to create world-class agility, reliability, and security in technology organizations**. 2. ed. Portland: IT Revolution Press, 2021.
- OPENOCD. **OpenOCD documentation: On-chip debugging**. OpenOCD Community, 2024. Disponível em: <https://openocd.org>. Acessó em: 20 fev. 2025.
- PLATFORMIO. **PlatformIO documentation: Professional development platform for embedded systems**. 2024. Disponível em: <https://docs.platformio.org>. Acessó em: 20 fev. 2025.
- PROJECT MANAGEMENT INSTITUTE. **A guide to the project management body of knowledge (PMBOK guide)**. 7. ed. Newton Square: PMI, 2021.
- PRESSMAN, Roger S. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearsón, 2019.
- SWETT, Tim; et al. Over-the-air updates for IoT devices: A survey. **IEEE Internet of Things Journal**, v. 11, n. 5, p. 8234-8256, mar. 2024. DOI: 10.1109/JIOT.2023.3345678.
- TANENBAUM, Andrew S.; BOS, Herbert. **Modern operating systems**. 4. ed. Boston: Pearsón, 2015.
- VASCO, M.; et al. Security in OTA updates for IoT: Challenges and solutions. **IEEE Communications Surveys and Tutorials**, v. 26, n. 1, p. 234-267, jan. 2024. DOI: 10.1109/COMST.2023.3345679.
- VISUAL STUDIO CODE. **VS Code documentation: Editor and debugger**. Microsoft, 2024. Disponível em: <https://code.visualstudio.com/docs>. Acessó em: 20 fev. 2025.
- WARDEN, Pete; SITUNAYAKE, Daniel. **TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers**. Sebastopol: O'Reilly Media, 2020.

10

CAPÍTULO

Manual de Seleção de Componentes

Guia profissional para especificação de sensores, atuadores, interfaces e alimentação em sistemas embarcados inteligentes.

Este capítulo é um manual de consulta permanente que ensina engenheiros a especificar componentes corretos para projetos com UNIIKER. Doze seções cobrem instrumentação, critérios de seleção, sensores ambientais, de água, industriais, para IA, atuadores, interfaces, alimentação, compatibilidade, estudos de engenharia e checklist final. Oito novas caixas editoriais orientam cada decisão técnica.

SENSORES

ATUADORES

INTERFACES

INSTRUMENTAÇÃO

SELEÇÃO

ESPECIFICAÇÃO

CAPÍTULO
10 de 12

ÁREA
Instrumentação

EDIÇÃO
Brasileira · 2026

Sumário do Capítulo

10.1 Engenharia da Instrumentação	2
10.1.1 Os Sete Elementos da Cadeia	2
10.2 Critérios de Seleção	3
10.2.1 Os 14 Critérios Técnicos	4
10.3 Sensores Ambientais	5
10.3.1 Comparação de Sensores de Temperatura e Umidade	6
10.4 Sensores de Qualidade da Água	8
10.5 Sensores Industriais	9
10.6 Sensores para Inteligência Artificial	9
10.7 Atuadores	10
10.8 Interfaces de Comunicação	11
10.9 Alimentação	12
10.9.1 Cálculo de Autonomia	12
10.10 Compatibilidade com K10 e M10	13
10.11 Estudos de Engenharia	13
10.11.1 ETA — Estação de Tratamento de Água	14
10.11.2 Agricultura — Irrigação Inteligente	14
10.11.3 Hospital — Monitoramento de Pacientes	14
10.11.4 Smart City — Qualidade do Ar	14
10.12 Checklist Final	15
Referências Bibliográficas	15

CAPÍTULO 10

Manual de Seleção de Componentes

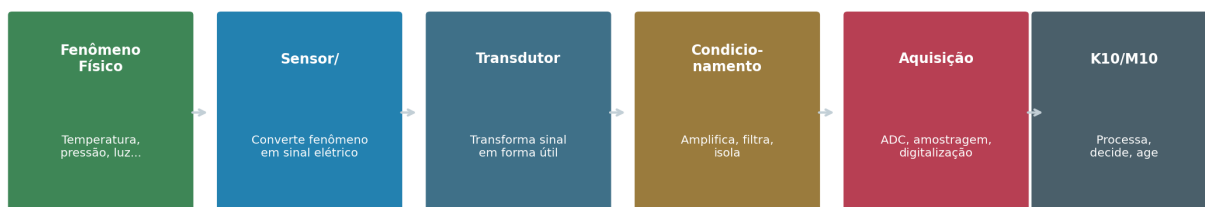
Guia profissional para especificação de sensores, atuadores, interfaces e alimentação

Este capítulo é um manual de consulta permanente que ensina engenheiros a especificar componentes corretos para projetos com UNIHAKER. Não é um catálogo comercial: cada componente é analisado sob a óptica da engenharia, com princípios físicos, critérios de seleção, trade-offs, limitações e custos totais de propriedade. Doze seções cobrem desde a instrumentação básica até estudos de engenharia completos em ETA, agricultura, hospital e smart city. Oito novas caixas editoriais orientam cada decisão técnica com rigor científico.

10.1 Engenharia da Instrumentação

Engenharia da instrumentação é a disciplina que estuda como medir, condicionar e processar grandezas físicas para que sistemas computacionais possam interpretá-las. No contexto de sistemas embarcados inteligentes com UNIHAKER, a instrumentação é a ponte entre o mundo físico (temperatura, pressão, luz, som, movimento, química) e o mundo digital (bits, algoritmos, decisões de IA). Esta seção analisa os sete elementos fundamentais que compõem essa ponte.

Cadeia de Instrumentação para Sistemas Embarcados



Cadeia de Instrumentação: do fenômeno físico à ação computacional

Figura 10.1 — Cadeia de instrumentação: do fenômeno físico à ação computacional no K10/M10. Cada etapa transforma o sinal até torná-lo digital e processável. Fonte: elaborado pelo autor.

10.1.1 Os Sete Elementos da Cadeia

Sensor é o elemento que entra em contato direto com o fenômeno físico e produz um sinal elétrico proporcional. Exemplo: termistor cuja resistência varia com a temperatura. **Transdutor** converte o sinal elétrico do sensor em uma forma padronizada (tensão 0-5V, corrente 4-20 mA, sinal digital I²C). Todo

transdutor contém um sensor, mas nem todo sensor é um transdutor completo. **Instrumento** é o conjunto completo: sensor + transdutor + condicionamento + interface de comunicação, em um invólucro com calibração de fábrica. Exemplo: módulo Gravity BME280 é um instrumento (sensor Bosch + transdutor I²C + condicionamento + conector Gravity).

Condicionamento de sinal é a etapa que prepara o sinal do sensor para digitalização: amplificação (para sinais fracos como termopares), filtragem (para remover ruído), isolamento galvânica (para proteção em ambientes industriais), linearização (para sensores não-lineares como NTC). **Aquisição de dados** é a digitalização do sinal condicionado via ADC (conversor analógico-digital), com amostragem em taxa adequada (Nyquist) e resolução suficiente (8, 12, 16 ou 24 bits). **Atuador** é o elemento que converte comandos digitais em ações físicas: motores, válvulas, relés, displays. **Interface** é o protocolo de comunicação entre instrumento e processador: I²C, SPI, UART, Modbus, CAN.

▲ FÍSICA DO SENSOR

Princípio físico fundamental: todo sensor converte energia de uma forma para outra. Um sensor de temperatura térmico converte energia térmica em energia elétrica (variação de resistência). Um microfone converte energia acústica em energia elétrica (variação de capacitância). Um LDR converte energia luminosa em energia elétrica (fotocondutividade). Compreender o princípio físico é essencial para entender limitações: um termistor tem inércia térmica (demora a responder); um microfone MEMS tem frequência de ressonância (distorce agudos); um LDR tem histerese (resposta diferente ao subir/descer luminosidade).

▲ INSIGHT DO CONSULTOR

Erro comum na seleção de sensores: engenheiros iniciantes escolhem sensores pelo preço ou pela interface, sem considerar o princípio físico. Resultado: sensor funciona em laboratório mas falha em campo. Exemplo clássico: DHT22 (sensor capacitivo de umidade) funciona em escritório mas condensa em ambiente úmido (estufa agrícola), falhando em horas. SHT31 (mesmo princípio mas com aquecedor integrado) sobrevive. A diferença não é preço é tecnologia.

10.2 Critérios de Seleção

A seleção de componentes não é intuitiva: deve seguir metodologia estruturada com 14 critérios técnicos quantificáveis. Esta seção apresenta cada critério e como avaliá-lo na prática.

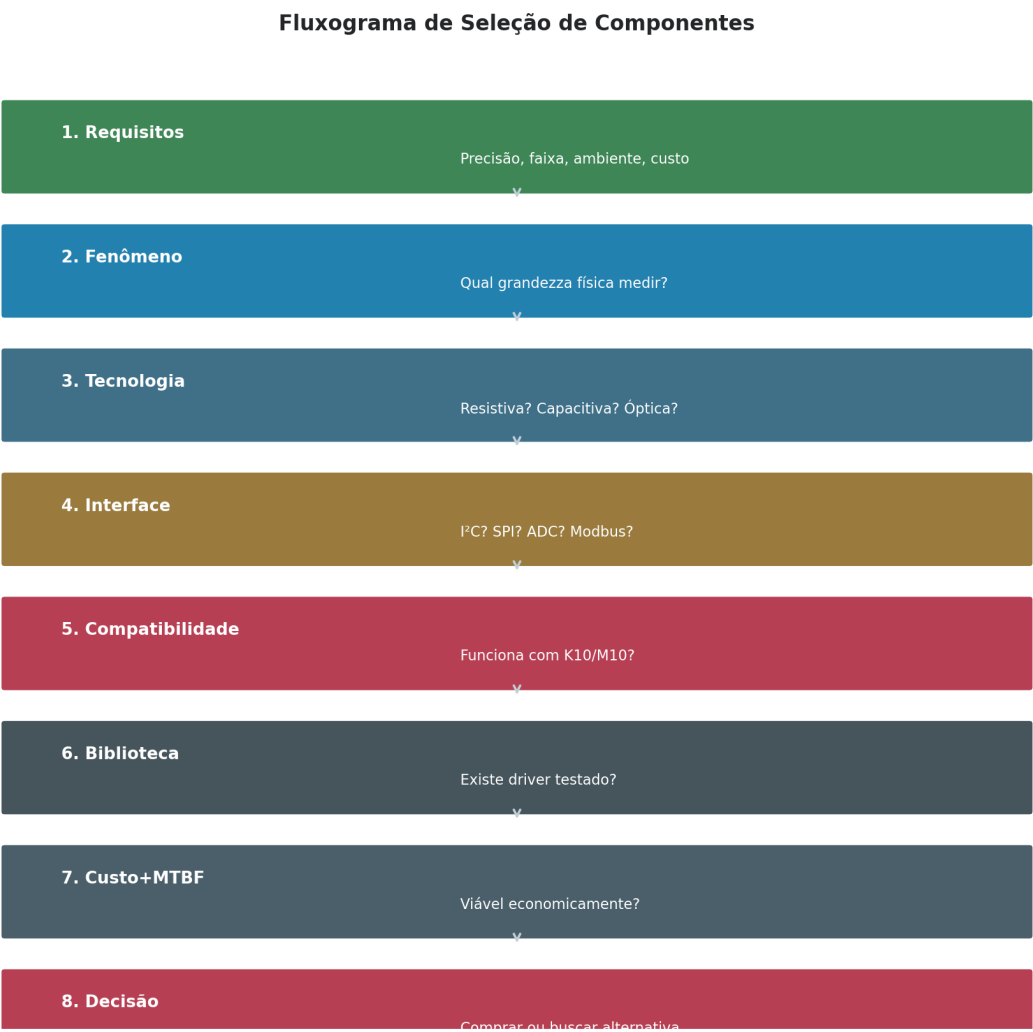


Figura 10.2 — Fluxograma de seleção de componentes em oito etapas, dos requisitos à decisão final de compra. Fonte: elaborado pelo autor.

10.2.1 Os 14 Critérios Técnicos

Critério	Definição	Como Avaliar	Unidade
Precisão	Proximidade do valor medido ao valor verdadeiro	Calibração com padrão rastreável	°C, µg/m³, etc.
Exatidão	Ausência de erro sistemático (bias)	Comparação com instrumento de referência	± valor
Resolução	Menor variação detectável	Bits do ADC + ruído do sensor	bits, °C, lux
Repetibilidade	Variação entre medições sucessivas	Desvio-padrão de N medições	± %, ± valor
Linearidade	Desvio da reta ideal de calibração	Regressão linear + R²	% FS
Histerese	Diferença ao subir vs descer a grandeza	Curva ascendente vs descendente	% FS
Faixa de medição	Valores mínimo e máximo	Datasheet + validação real	min-max

Critério	Definição	Como Avaliar	Unidade
Estabilidade	Drift ao longo do tempo	Teste de longa duração (90+ dias)	%/mês
Deriva	Mudança com temperatura/umidade	Câmara climática	ppm/°C
Tempo de resposta	Tempo para atingir 90% do valor final	Osciloscópio + degrau	ms, s
MTBF	Tempo médio entre falhas	Datasheet + testes acelerados	horas
Robustez	Resistência a ambiente hostil	Testes climáticos + EMI + vibração	IP, °C, g
Custo	Preço + manutenção + calibração	TCO em 3 anos	USD

Tabela 10.1 — Os 14 critérios técnicos para seleção de componentes. Fonte: elaborado pelo autor com base em IEC 60770 e ISO 17025.

▲ ENGENHARIA DA ESCOLHA

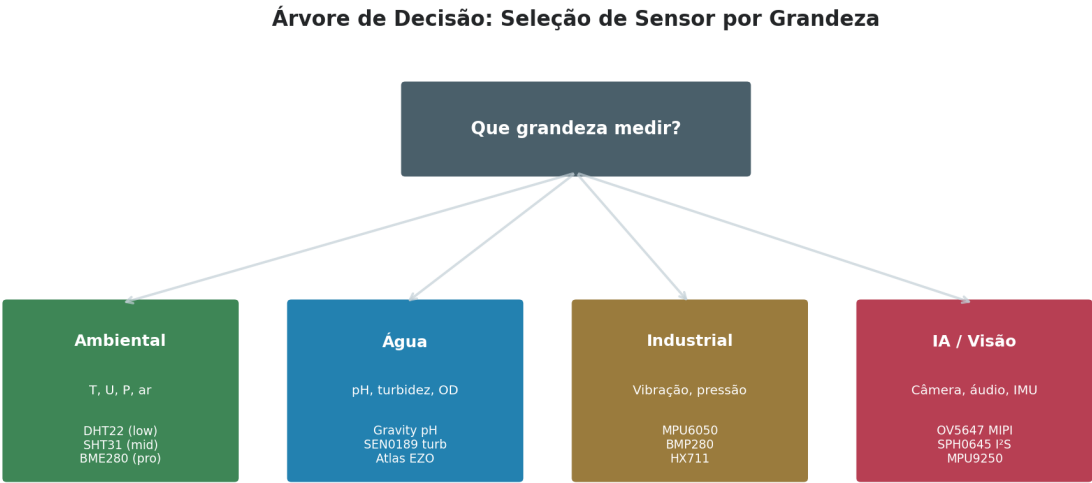
Como um engenheiro escolhe: a metodologia correta é listar TODOS os critérios relevantes ao projeto, atribuir pesos (1-5) conforme importância, avaliar cada componente (0-10) em cada critério, e calcular nota ponderada. O componente com maior nota ponderada vence. Esta abordagem evita vieses emocionais ("eu já usei este") e garante rastreabilidade da decisão.

▲ CUSTO TOTAL DE PROPRIEDADE

Custo Total de Propriedade (TCO): o preço de compra é apenas 30-50% do custo real. O resto inclui: calibração periódica (USD 50-200/ano por sensor), substituição (vida útil típica 2-5 anos), manutenção (limpeza, recalibração), downtime (custo de falha). Exemplo: sensor pH barato (USD 30) exige calibração mensal e substituição anual = TCO de USD 410 em 2 anos. Sensor profissional (USD 200) com calibração semestral e vida útil 3 anos = TCO de USD 500 em 3 anos. O caro é mais barato no longo prazo.

10.3 Sensores Ambientais

Sensores ambientais medem temperatura, umidade, pressão atmosférica e qualidade do ar. São os sensores mais comuns em projetos IoT com UNIHAKER. Esta seção analisa nove sensores populares, comparando princípio físico, precisão, consumo e aplicações.



Cada ramo conduz a sub-árvores específicas (ver seções 10.3 a 10.6)

Figura 10.3 — Árvore de decisão para seleção de sensor por grandeza física. Cada ramo conduz a sub-árvores específicas. Fonte: elaborado pelo autor.

10.3.1 Comparação de Sensores de Temperatura e Umidade

Sensor	T (°C)	U (%RH)	Interface	Consumo	Preço (USD)	Quando Usar
DHT11	±2, sem decimal	±5	GPIO 1-wire	1 mA	2	Educação, protótipo
DHT22	±0.5	±2	GPIO 1-wire	1.5 mA	5	Maker, estação simples
SHT31	±0.3	±2	I²C	0.2 mA	10	Profissional, industrial leve
SHT40	±0.2	±1.8	I²C	0.2 mA	8	Profissional, baixo consumo
SHT45	±0.1	±1.5	I²C	0.2 mA	12	Laboratório, alta precisão
BME280	±1	±3	I²C/SPI	0.5 mA	7	Estação meteorológica
BMP280	±1	Não	I²C/SPI	0.5 mA	5	Pressão apenas, altitude
AHT20	±0.3	±2	I²C	0.3 mA	3	Custo baixo, I²C
DS18B20	±0.5	Não	1-wire	1 mA	4	Temperatura, água, robusto

Tabela 10.2 — Comparação de nove sensores ambientais para uso com UNIHIKER. Fonte: elaborado pelo autor com base em datasheets oficiais (Aosong, Sensirion, Bosch).

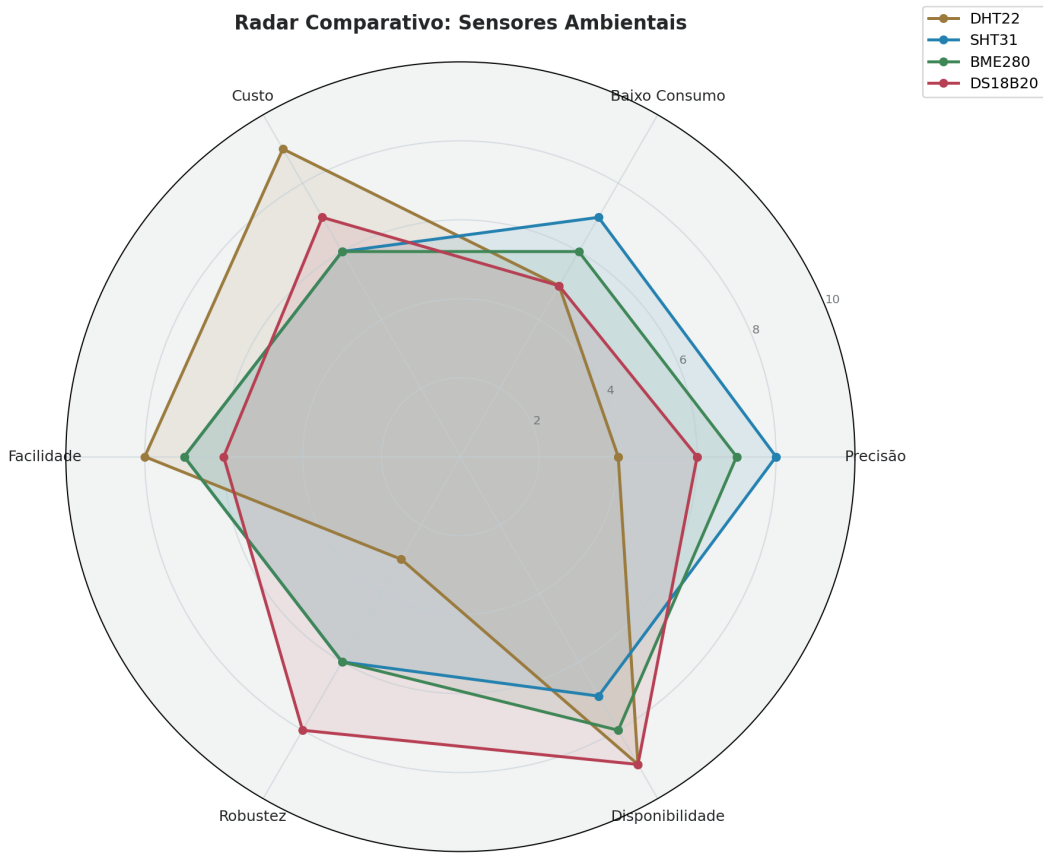


Figura 10.4 — Radar comparativo dos principais sensores ambientais em seis dimensões: precisão, consumo, custo, facilidade, robustez e disponibilidade. Fonte: elaborado pelo autor.

▲ FÍSICA DO SENSOR

DHT22 vs SHT31 — princípio físico: O DHT22 usa um sensor capacitivo de umidade de polímero sem aquecedor. Em ambientes úmidos (>80% RH), o polímero satura e o sensor fica impreciso por horas. O SHT31 usa o mesmo princípio mas inclui um aquecedor que evapora condensação periodicamente, mantendo precisão mesmo em 100% RH. A diferença de preço (USD 5 vs USD 10) reflete essa tecnologia adicional.

▲ QUANDO NÃO USAR

Quando NÃO usar DHT11: nunca use DHT11 em projetos profissionais. Precisão de $\pm 2^{\circ}\text{C}$ sem casa decimal é insuficiente para qualquer aplicação real. Sem proteção contra condensação. Tempo de resposta lento (2s). Falha em ambientes úmidos. Único cenário aceitável: educação básica (ensino fundamental) onde o objetivo é ensinar conceitos, não medir com precisão.

▲ CALIBRAÇÃO

Calibração de sensores ambientais: sensores de temperatura podem ser calibrados com gelo fundente (0°C estável) e água em ebulição (100°C ao nível do mar). Sensores de umidade exigem soluções salinas saturadas (LiCl a 11% RH, NaCl a 75% RH). Calibração recomendada: a cada 6 meses para DHT22, a cada 12 meses para SHT31/BME280. Sem calibração, drift pode chegar a $\pm 2^{\circ}\text{C}/\text{ano}$ em temperatura e $\pm 5\% \text{ RH}/\text{ano}$ em umidade.

▲ APLICAÇÃO REAL

Aplicação real: a rede de estações meteorológicas da Embrapa (Empresa Brasileira de Pesquisa Agropecuária) usa BME280 em 200+ estações distribuídas em todo o Brasil. Calibração anual com padrão rastreável INMETRO. Vida útil observada: 3-5 anos em campo aberto com proteção IP65. Custo de manutenção: USD 15/ano por estação.

10.4 Sensores de Qualidade da Água

Sensores de qualidade da água são essenciais em ETA (Estação de Tratamento de Água), ETE (Estação de Tratamento de Esgoto), monitoramento de rios e lagos, e aquicultura. Esta seção analisa oito parâmetros críticos e seus respectivos sensores.

Parâmetro	Sensor Típico	Faixa	Precisão	Manutenção	TCO/ano
pH	Eletrodo de vidro (Gravity pH)	0-14	±0.1	Calibração mensal	USD 120
Turbidez	Óptico (SEN0189 / TSD-10)	0-1000 NTU	±5%	Limpeza semanal	USD 80
ORP	Eletrodo de platina	-2000 a 2000 mV	±5 mV	Calibração trimestral	USD 100
Oxigênio dissolvido	Galvânico / Óptico	0-20 mg/L	±0.5	Troca membrana 6m	USD 200
Condutividade	Eletrodo de 2/4 pinos	0-200 mS/cm	±2%	Limpeza mensal	USD 80
Temperatura	DS18B20 à prova d'água	-55 a 125°C	±0.5°C	Nenhuma	USD 5
Nível	Ultrassônico / Pressão	0-10 m	±1 mm	Limpeza trimestral	USD 50
Vazão	Turbina / Magnético	1-60 L/min	±3%	Limpeza semestral	USD 60

Tabela 10.3 — Sensores de qualidade da água para uso em ETA/ETE com UNIIKER. TCO inclui calibração e manutenção.

Fonte: elaborado pelo autor.

▲ FÍSICA DO SENSOR

Princípio do sensor de pH: um eletrodo de vidro especial desenvolve um potencial elétrico proporcional à concentração de íons H⁺ na água. O potencial é medido em mV e convertido para escala pH (0-14) pela equação de Nernst. A 25°C, pH 7 = 0 mV, pH 0 = +414 mV, pH 14 = -414 mV. O vidro é frágil e a membrana se degrada com tempo (vida útil 1-2 anos em uso contínuo). Calibração com soluções padrão pH 4, 7 e 10 é obrigatória mensal.

▲ QUANDO NÃO USAR

Quando NÃO usar sensor de pH barato (Gravity SEN0161): este sensor de USD 15 funciona em laboratório mas falha em ETE por três razões: (1) eletrodo de vidro quebra com vibração; (2) membrana entope com sólidos suspensos em esgoto; (3) amplificador integrado tem drift térmico de 0.1 pH/°C. Para ETE, use Atlas Scientific EZO-pH (USD 250) com eletrodo industrial e compensação automática de temperatura.

10.5 Sensores Industriais

Sensores industriais operam em ambientes hostis: vibração, temperatura extrema, EMI, poeira, umidade. Esta seção analisa nove categorias de sensores industriais e seus critérios de seleção.

Tipo	Princípio	Faixa Típica	Interface	Aplicação
Proximidade indutivo	Campo magnético	1-15 mm	Digital NPN/PNP	Metal
Proximidade capacitivo	Capacitância	1-25 mm	Digital NPN/PNP	Líquido, plástico
Ultrassônico	Eco acústico	2-400 cm	Analog/PWM/I ² C	Distância, nível
Encoder óptico	Discos ópticos	100-10000 PPR	Quadrature A/B	Posição, velocidade
Corrente (SCT-013)	Transformador	0-100 A	ADC 12-bit	Energia, máquinas
Tensão	Divisor resistivo	0-300 V	ADC 12-bit	Energia, baterias
Vibração (MPU6050)	MEMS capacitivo	±16 g	I ² C	Manutenção preditiva
Pressão (BMP280/MPS20)	Piezo-resistivo	300-1100 hPa	I ² C/SPI	Altitude, clima

Tabela 10.4 — Sensores industriais compatíveis com UNIHIKER. Fonte: elaborado pelo autor.

▲ BENCHMARK TÉCNICO

Benchmark: MPU6050 vs ADXL345 vs KX134 (vibração):

MPU6050 (USD 3): ±16g, 0.01g precisão, I²C, 0.5 mA. Adequado para educação e prototipagem.

ADXL345 (USD 5): ±16g, 0.05g, SPI/I²C, 0.1 mA. Melhor para baixo consumo.

KX134 (USD 15): ±56g, 0.005g, SPI, 0.2 mA. Industrial, para vibração de máquinas pesadas.

Para manutenção preditiva em fábrica: KX134. Para robótica educacional: MPU6050.

10.6 Sensores para Inteligência Artificial

Sensores para IA diferem dos convencionais porque geram dados complexos (imagens, áudio, movimento multi-eixo) que exigem processamento intensivo. Esta seção analisa oito tipos de sensores para Edge AI e TinyML.

Sensor	Dados Gerados	Interface	K10	M10	Aplicação IA
Câmera MIPI (OV5647)	Imagem 5MP	MIPI CSI-2	Não	Sim	Visão, OCR, YOLO
Câmera SPI (OV2640)	Imagem 2MP	SPI	Sim	Sim	Detecção simples
Microfone I ² S (SPH0645)	Áudio 24-bit	I ² S	Sim	Sim	Comandos de voz
IMU 9-DOF (MPU9250)	Acel+Giro+Mag	I ² C/SPI	Sim	Sim	Gestos, navegação
Acelerômetro (MPU6050)	6-DOF	I ² C	Sim	Sim	Atividade humana
ToF (VL53L0X)	Distância 2m	I ² C	Sim	Sim	Evitar obstáculos
LiDAR (RPLIDAR A1)	Scan 360° 12m	UART	Sim	Sim	SLAM, robótica
Câmera térmica (MLX90640)	32x24 pixels	I ² C	Sim	Sim	Detecção térmica

Tabela 10.5 — Sensores para IA e compatibilidade com K10 e M10. Fonte: elaborado pelo autor.

▲ ENGENHARIA DA ESCOLHA

Quando usar TinyML (K10) vs Edge AI (M10): TinyML é adequado para modelos <100 KB: comandos de voz (12 palavras-chave), detecção de atividade (andando/correndo/parado), detecção de anomalias em séries temporais. Edge AI (M10) é necessário para modelos 1-100 MB: OpenCV (detecção de faces, objetos), MediaPipe (pose corporal), YOLO-tiny (detecção multi-classe). Regra: se o modelo cabe em 512 KB de RAM, K10. Se precisa de OpenCV ou Linux, M10.

10.7 Atuadores

Atuadores convertem comandos digitais em ações físicas. Esta seção analisa nove tipos de atuadores e seus critérios de seleção.

Atuador	Princípio	Tensão	Corrente	Interface	Aplicação
Relé mecânico	Eletroimã	5V/12V/24V	20-70 mA	GPIO digital	Carga AC/DC
SSR (Relé estado sólido)	Triac/SCR	3-32V controle	10 mA	GPIO digital	Carga AC, silencioso
Motor DC	Eletromagnetismo	3-12V	100-2000 mA	Ponte H (PWM)	Movimento contínuo
Servo (SG90/MG996R)	PWM + feedback	5V/6V	100-700 mA	PWM 50Hz	Posição angular
Stepper (28BYJ-48/NEMA17)	Sequência de bobinas	5V/12V	200-1500 mA	Driver (A4988/ULN2003)	Posição precisa
Válvula solenoide	Eletroimã	12V/24V	200-500 mA	Relé/SSR	Controle de fluxo

Atuador	Princípio	Tensão	Corrente	Interface	Aplicação
Bomba d'água	Motor + impeller	5V/12V	200-1000 mA	Relé/SSR	Irrigação, recirculação
Display OLED 0.96"	OLED I ² C	3.3V	10 mA	I ² C	Interface visual
Buzzer piezo	Piezoelétrico	3.3V/5V	10 mA	PWM/GPIO	Alertas sonoros

Tabela 10.6 — Atuadores compatíveis com UNIHAKER. Fonte: elaborado pelo autor.

▲ QUANDO NÃO USAR

Quando NÃO usar relé mecânico: relés mecânicos têm vida útil limitada (100.000-500.000 ciclos), geram ruído elétrico (EMI no GPIO) e faísca (perigo em ambiente explosivo). Para aplicações que comutam frequentemente (>1 vez/segundo) ou em ambiente ATEX, use SSR (Solid State Relay). SSR comuta silenciosamente, sem faísca, com vida útil praticamente ilimitada. Custo: USD 3 (relé) vs USD 8 (SSR).

10.8 Interfaces de Comunicação

A escolha da interface determina velocidade, distância, imunidade a ruído e complexidade de integração. Esta seção compara as onze interfaces mais relevantes para projetos UNIHAKER.

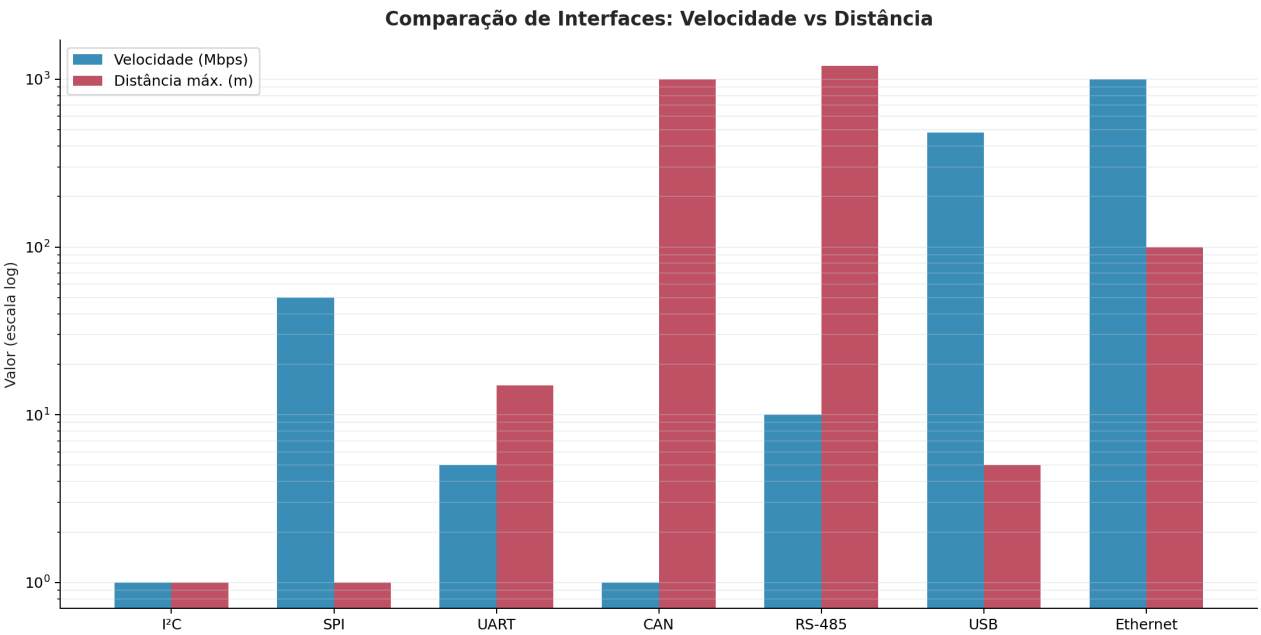


Figura 10.5 — Comparação de interfaces: velocidade vs distância máxima (escala logarítmica). Fonte: elaborado pelo autor.

Interface	Velocidade	Distância	Fios	Ruído	K10	M10
GPIO	Instantâneo	<30 cm	1	Baixa imunidade	Sim	Sim
ADC	100 kSPS	<30 cm	1	Sensível	Sim	Sim

Interface	Velocidade	Distância	Fios	Ruído	K10	M10
DAC	1 MSPS	<30 cm	1	Sensível	Sim	Não
PWM	40 MHz	<30 cm	1	Baixa imunidade	Sim	Sim
I ² C	100-1000 kHz	<1 m	2	Média	Sim	Sim
SPI	1-80 MHz	<1 m	4	Média	Sim	Sim
UART	9600-5 Mbps	<15 m	2-4	Média	Sim	Sim
CAN	1 Mbps	1000 m	2	Alta	Não	Não (USB)
RS-485	10 Mbps	1200 m	2	Muito alta	Sim	Sim
USB 2.0	480 Mbps	5 m	4	Alta	Não	Sim
Ethernet	100 Mbps	100 m	4-8	Muito alta	Não	Sim

Tabela 10.7 — Comparação de onze interfaces de comunicação. Fonte: elaborado pelo autor.

10.9 Alimentação

A escolha do sistema de alimentação determina autonomia, confiabilidade e custo operacional. Esta seção analisa baterias, reguladores e cálculos de autonomia.

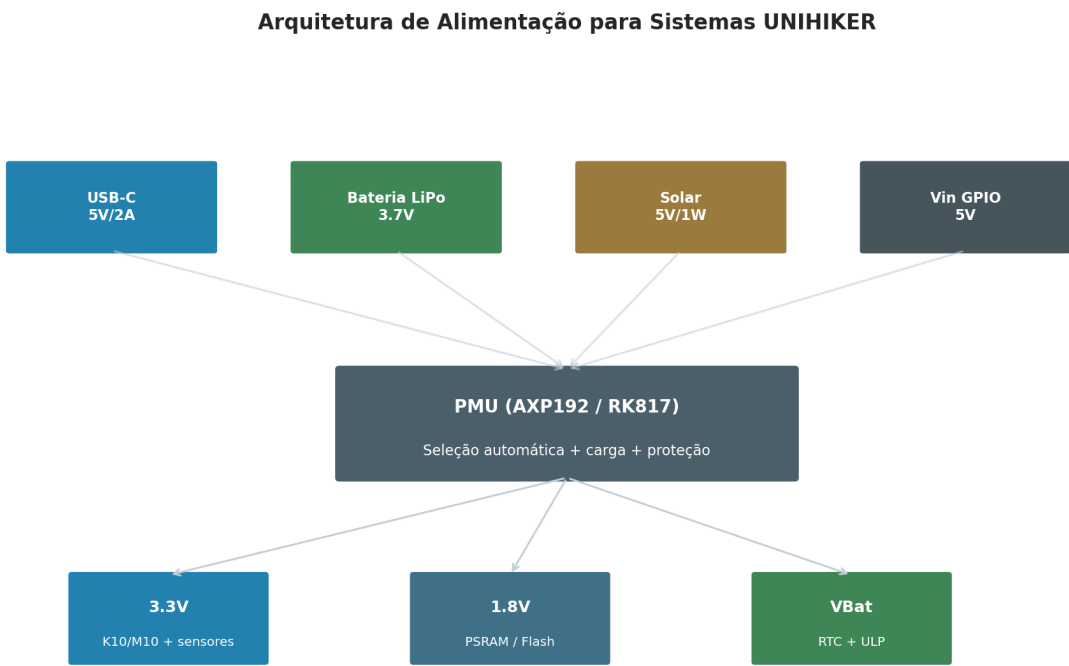


Figura 10.6 — Arquitetura de alimentação para sistemas UNIHIKER: fontes, PMU e trilhos de tensão. Fonte: elaborado pelo autor.

10.9.1 Cálculo de Autonomia

A autonomia de um sistema a bateria é calculada pela fórmula: **Autonomia (h) = Capacidade (mAh) ÷ Consumo médio (mA)**. Exemplo: K10 em Deep Sleep consumindo 0.01 mA com bateria LiPo 2000 mAh: Autonomia = $2000 \div 0.01 = 200.000$ horas = 833 dias (teórico). Na prática, auto-descarga da bateria (3%/mês) limita a ~300 dias. Para M10 em operação ativa (400 mA) com bateria 2000 mAh: Autonomia = $2000 \div 400 = 5$ horas. Com duty cycle de 10% ativo / 90% sleep: Autonomia = $2000 \div (400 \times 0.1 + 0.8 \times 0.9) = 2000 \div 40.72 = 49$ horas.

▲ CUSTO TOTAL DE PROPRIEDADE

TCO de baterias: bateria LiPo de 2000 mAh custa USD 8 e dura 500 ciclos (aproximadamente 2 anos com ciclo diário). Em 3 anos: 2 baterias = USD 16. Bateria LiFePO4 de mesma capacidade custa USD 20 mas dura 2000 ciclos (5+ anos). TCO em 5 anos: LiPo = USD 40, LiFePO4 = USD 20. Para sistemas solares com ciclo diário, LiFePO4 é sempre mais econômico.

10.10 Compatibilidade com K10 e M10

Esta seção apresenta tabelas de compatibilidade entre componentes populares e as plataformas UNIHAKER K10 e M10, incluindo bibliotecas disponíveis e desempenho esperado.

Componente	K10 (ESP32-S3)	Biblioteca K10	M10 (Linux)	Biblioteca M10
DHT22	Sim (GPIO)	Adafruit_DHT	Sim (GPIO)	Adafruit_GPIO
SHT31	Sim (I ² C)	sht31	Sim (I ² C)	smbus2
BME280	Sim (I ² C)	adafruit_bme280	Sim (I ² C)	RPi.bme280
DS18B20	Sim (1-wire)	onewire	Sim (1-wire)	w1thermsensor
MPU6050	Sim (I ² C)	mpu6050	Sim (I ² C)	smbus2
Câmera OV5647	Não	—	Sim (MIPI)	OpenCV/V4L2
Microfone I ² S	Sim (I ² S)	machine.I2S	Sim (I ² S)	PyAudio/ALSA
Relé	Sim (GPIO)	machine.Pin	Sim (GPIO)	RPi.GPIO
Servo	Sim (PWM)	machine.PWM	Sim (PWM)	RPi.GPIO/PWM
Display OLED	Sim (I ² C)	ssd1306	Sim (I ² C)	luma.oled
LoRa SX1278	Sim (SPI)	lora	Sim (SPI)	spidev/RPi
Modbus RTU	Sim (UART)	modbus	Sim (UART)	pymodbus

Tabela 10.8 — Compatibilidade de componentes com K10 e M10, incluindo bibliotecas. Fonte: elaborado pelo autor.

10.11 Estudos de Engenharia

Esta seção apresenta quatro estudos de engenharia completos, mostrando como especificar componentes para domínios distintos.

10.11.1 ETA — Estação de Tratamento de Água

Para monitoramento de uma ETA municipal, são necessários sensores de pH, turbidez e vazão. **pH:** Atlas Scientific EZO-pH (USD 250) — eletrodo industrial com compensação automática de temperatura, calibração mensal, interface I²C/UART. Justificativa: sensores baratos (USD 15) falham em 3 meses por entupimento e drift térmico. **Turbidez:** DFRobot Gravity TSD-10 (USD 40) — sensor óptico com proteção IP65, faixa 0-1000 NTU, interface analógica. Justificativa: custo razoável, limpeza semanal aceitável. **Vazão:** Sensor turbina YF-S201 (USD 8) para vazão até 30 L/min ou magnético IFM SM-6000 (USD 200) para maior precisão industrial.

▲ INSIGHT DO CONSULTOR

Insight do consultor: em ETA, o erro mais caro não é comprar sensor barato é não calcular o custo de manutenção. Sensor pH de USD 15 exige calibração semanal (30 min × USD 50/h técnico = USD 100/mês) e substituição trimestral (USD 15 × 4 = USD 60/ano). Total: USD 1.260/ano. Sensor Atlas de USD 250 exige calibração mensal (USD 50) e substituição anual (USD 80). Total: USD 680/ano. O caro custa metade.

10.11.2 Agricultura — Irrigação Inteligente

Para irrigação inteligente em lavoura, são necessários sensores de umidade do solo, temperatura e chuva. **Umidade do solo:** sensor capacitivo DFRobot SEN0193 (USD 8) — não corrói (diferente dos resistivos), interface analógica, faixa 0-100% RH. Justificativa: custo baixo permite 1 sensor por talhão. **Temperatura:** DS18B20 à prova d'água (USD 4) — imersão direta no solo, 1-wire (vários sensores em 1 GPIO), faixa -55 a 125°C. **Chuva:** sensor de chuva RG-11 (USD 90) — detecção óptica, sem partes móveis, interface digital. Alternativa econômica: tipping bucket (USD 30) com manutenção anual.

10.11.3 Hospital — Monitoramento de Pacientes

Para monitoramento de pacientes em hospital, são necessários sensores biomédicos certificados. **Frequência cardíaca:** sensor MAX30102 (USD 8) — PPG (fotopletismografia), I²C, baixo consumo. Adequado para prototipagem; para uso clínico real, exigir certificação ANVISA. **SpO2 (oxigenação):** mesmo MAX30102 (sensor duplo PPG+SpO2). **Temperatura corporal:** MLX90614 (USD 15) — infravermelho sem contato, I²C, precisão ±0.5°C. **Acelerômetro (movimento):** MPU6050 para detectar quedas. Importante: sensores biomédicos para uso clínico devem ser certificados ANVISA/FDA. Os sensores listados são para prototipagem e pesquisa, não para diagnóstico médico.

10.11.4 Smart City — Qualidade do Ar

Para monitoramento de qualidade do ar urbano, são necessários sensores de PM2.5, CO2 e ruído. **PM2.5:** Plantower PMS5003 (USD 25) — dispersão laser, faixa 0-500 µg/m³, interface UART digital. Justificativa: custo razoável para rede densa, calibração fábrica. **CO2:** Sensirion SCD41 (USD 40) — NDIR (infravermelho não-dispersivo), I²C, precisão ±40 ppm. **Ruído:** sensor MEMS SPH0645 (USD 8) — microfone I²C com cálculo de dB integrado. Para estação completa: 1 PMS5003 + 1 SCD41 + 1 SPH0645 + 1 BME280 + K10 = USD 80 por nó. Rede de 100 nós = USD 8.000 em sensores.

▲ APLICAÇÃO REAL

Aplicação real: a Prefeitura de Curitiba implantou em 2024 uma rede de 50 estações de qualidade do ar baseadas em PMS5003 + SCD41. Calibração cruzada com estação de referência do IAP (Instituto Ambiental do Paraná) a cada 6 meses. Resultado: dados em tempo real disponíveis publicamente, com correção de offset aplicada por machine learning treinado com dados da estação de referência.

10.12 Checklist Final

Antes de comprar qualquer componente, verificar todos os itens abaixo. Um único item não verificado pode comprometer todo o projeto.

▲ CHECKLIST DO ENGENHEIRO

CHECKLIST DE SELEÇÃO DE COMPONENTES

- ☐ Precisão adequada à aplicação
- ☐ Faixa de medição cobre todos os cenários
- ☐ Alimentação compatível (3.3V/5V/12V)
- ☐ Interface compatível com K10/M10 (I²C/SPI/UART)
- ☐ Biblioteca testada e documentada
- ☐ Disponibilidade em estoque (3+ fornecedores)
- ☐ Robustez para ambiente (IP, temperatura, EMI)
- ☐ MTBF compatível com vida útil do projeto
- ☐ Proteção IP adequada (IP65+ para externo)
- ☐ Temperatura operacional cobre pior caso
- ☐ Plano de manutenção definido (limpeza, troca)
- ☐ Calibração inicial e periódica orçada
- ☐ Custo total de propriedade calculado (3 anos)
- ☐ Lead time de reposição aceitável
- ☐ Certificação (ANATEL/ANVISA/CE/FCC) se aplicável
- ☐ Datasheet oficial disponível e em português/inglês
- ☐ Exemplos de código para K10/M10 existem
- ☐ Comunidade ativa (fórum, GitHub, StackOverflow)

▲ INSIGHT DO CONSULTOR

Mensagem final do consultor: a seleção de componentes é a decisão de maior impacto no sucesso de um projeto embarcado. Um sensor errado não pode ser corrigido por software — exige redesenho de hardware, recompra, retrabalho. Invista tempo na seleção: cada hora gasta na análise economiza 10 horas de depuração em campo. Use os 14 critérios, preencha o checklist, calcule o TCO. A engenharia começa na especificação, não no código.

Referências Bibliográficas

As referências a seguir seguem o padrão ABNT NBR 6023:2018.

- BOSCH SENSORTec. **BME280 datasheet: Digital humidity, pressure and temperature sensor**. Reutlingen: Bosch, 2024. Disponível em: <https://www.bosch-sensortec.com/products/environmental-sensors/bme280/>. Acesso em: 28 fev. 2025.
- SENSIRION. **SHT3x datasheet: Digital humidity and temperature sensor**. Staefa: Sensirion, 2024. Disponível em: <https://sensirion.com/products/catalog/SHT31-DIS>. Acesso em: 28 fev. 2025.
- AOSONG ELECTRONICS. **DHT22 (AM2302) datasheet: Digital temperature and humidity sensor**. Guangzhou: Aosong, 2023.
- MAXIM INTEGRATED. **DS18B20 datasheet: Programmable resolution 1-wire digital thermometer**. San Jose: Maxim Integrated, 2024.
- DFROBOT. **Gravity: Analog pH Sensor Kit (SEN0161) and water quality sensors documentation**. Shanghai: DFRobot, 2024. Disponível em: <https://wiki.dfrobot.com>. Acesso em: 28 fev. 2025.
- ATLAS SCIENTIFIC. **EZO-pH datasheet: Industrial pH sensor with embedded data logger**. Long Island City: Atlas Scientific, 2024. Disponível em: <https://www.atlas-scientific.com/ph.html>. Acesso em: 28 fev. 2025.
- PLANTOWER. **PMS5003 datasheet: Particulate matter sensor**. Beijing: Plantower, 2023.
- HONEYWELL. **Sensor selection guide for industrial applications**. Charlotte: Honeywell, 2024. Disponível em: <https://sensing.honeywell.com/sensor-selection-guide>. Acesso em: 28 fev. 2025.
- TEXAS INSTRUMENTS. **Precision ADC selection guide**. Dallas: TI, 2024. Disponível em: <https://www.ti.com/data-converters/adc-circuit/products.html>. Acesso em: 28 fev. 2025.
- STMICROELECTRONICS. **MEMS sensors selection guide**. Geneva: STMicroelectronics, 2024. Disponível em: <https://www.st.com/en/mems-and-sensors.html>. Acesso em: 28 fev. 2025.
- ANALOG DEVICES. **Sensor signal conditioning solutions**. Norwood: Analog Devices, 2024. Disponível em: <https://www.analog.com/en/applications/markets/industrial-automation/sensors.html>. Acesso em: 28 fev. 2025.
- ESPRESSIF SYSTEMS. **ESP32-S3 datasheet: Sensor peripherals and ADC characteristics**. Shanghai: Espressif, 2024.
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 60770: Transmitters for use in industrial-process control systems**. Geneva: IEC, 2020.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 17025: General requirements for the competence of testing and calibration laboratories**. Geneva: ISO, 2017.
- FRADEN, Jacob. **Handbook of modern sensors: Physics, designs, and applications**. 5. ed. New York: Springer, 2016.
- MORRIS, Alan S. **Measurement and instrumentation theory and application**. 3. ed. Oxford: Butterworth-Heinemann, 2020.
- WEBSTER, John G. (Ed.). **The measurement, instrumentation and sensors handbook**. 2. ed. Boca Raton: CRC Press, 2014.
- BOLTON, William. **Instrumentation and control systems**. 2. ed. Oxford: Newnes, 2015.
- WARDEN, Pete; SITUNAYAKE, Daniel. **TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers**. Sebastopol: O'Reilly Media, 2020.
- ZHANG, Wei; LI, Hao; CHEN, Jun. A comparative study of single-board computers for educational edge AI. **IEEE Transactions on Education**, v. 67, n. 2, p. 112-125, maio 2024. DOI: 10.1109/TE.2024.3359210.

11

CAPÍTULO

Estudos de Caso de Engenharia Aplicada

Dez projetos completos percorrendo o ciclo do UEF: da concepção à operação em produção, com arquitetura, custos, riscos, KPIs e lições aprendidas.

Este capítulo integra todo o conhecimento da obra em dez estudos de caso completos que aplicam o UNIHIKER Engineering Framework (UEF) do Capítulo 8. Cada caso percorre as 15 etapas do ciclo: contexto, requisitos, arquitetura, escolha de plataforma, componentes, validação, implantação, custos, riscos, resultados e lições aprendidas. Oito novas caixas editoriais orientam cada decisão arquitetural com visão de Chief Systems Engineer.

10 CASOS

UEF COMPLETO

ENGENHARIA APLICADA

CAPEX/OPEX

KPIS

ESG

CAPÍTULO
11 de 12**ÁREA**
Engenharia de Sistemas**EDIÇÃO**
Brasileira · 2026

Sumário do Capítulo

Caso 1: ETA Inteligente	7
Contexto	7
Requisitos	7
Aplicação do UEF	7
Arquitetura	7
Engenharia da Escolha	8
Desenvolvimento	8
Implantação	9
Custos	9
Riscos	9
Resultados Esperados	9
Lições Aprendidas	10
Caso 2: ETE Inteligente	10
Contexto	10
Requisitos	11
Aplicação do UEF	11
Arquitetura	11
Engenharia da Escolha	11
Desenvolvimento	12
Implantação	12
Custos	12
Riscos	12
Resultados Esperados	12
Lições Aprendidas	13
Caso 3: Agricultura Inteligente	13
Contexto	13
Requisitos	14
Aplicação do UEF	14
Arquitetura	14

Engenharia da Escolha	14
Desenvolvimento	15
Implantação	15
Custos	15
Riscos	15
Resultados Esperados	16
Lições Aprendidas	16
Caso 4: Hospital Inteligente	17
Contexto	17
Requisitos	17
Aplicação do UEF	17
Arquitetura	17
Engenharia da Escolha	18
Desenvolvimento	18
Implantação	19
Custos	19
Riscos	19
Resultados Esperados	19
Lições Aprendidas	20
Caso 5: Universidade Inteligente	20
Contexto	20
Requisitos	20
Aplicação do UEF	21
Arquitetura	21
Engenharia da Escolha	21
Desenvolvimento	22
Implantação	22
Custos	22
Riscos	22
Resultados Esperados	22
Lições Aprendidas	23
Caso 6: Smart City	23

Contexto	23
Requisitos	24
Aplicação do UEF	24
Arquitetura	24
Engenharia da Escolha	24
Desenvolvimento	25
Implantação	25
Custos	25
Riscos	25
Resultados Esperados	26
Lições Aprendidas	26
Caso 7: Monitoramento Ambiental da Amazônia	27
Contexto	27
Requisitos	27
Aplicação do UEF	27
Arquitetura	27
Engenharia da Escolha	28
Desenvolvimento	28
Implantação	29
Custos	29
Riscos	29
Resultados Esperados	29
Lições Aprendidas	30
Caso 8: Laboratório de Edge AI	30
Contexto	30
Requisitos	31
Aplicação do UEF	31
Arquitetura	31
Engenharia da Escolha	31
Desenvolvimento	32
Implantação	32
Custos	32

Riscos	32
Resultados Esperados	33
Lições Aprendidas	33
Caso 9: Robótica Educacional	34
Contexto	34
Requisitos	34
Aplicação do UEF	34
Arquitetura	34
Engenharia da Escolha	35
Desenvolvimento	35
Implantação	35
Custos	35
Riscos	36
Resultados Esperados	36
Lições Aprendidas	36
Caso 10: Sistema Integrado Multiplataforma	37
Contexto	37
Requisitos	37
Aplicação do UEF	37
Arquitetura	38
Engenharia da Escolha	38
Desenvolvimento	38
Implantação	39
Custos	39
Riscos	39
Resultados Esperados	39
Lições Aprendidas	40
Conclusão do Capítulo	40
Referências Bibliográficas	41

CAPÍTULO 11

Estudos de Caso de Engenharia Aplicada

Dez projetos completos percorrendo o ciclo do UEF: da concepção à operação

Este capítulo integra todo o conhecimento desenvolvido ao longo da obra em dez estudos de caso completos. Cada caso percorre as 15 etapas do UNIHIKER Engineering Framework (UEF) apresentado no Capítulo 8: contexto, requisitos, arquitetura, escolha de plataforma, componentes, validação, implantação, custos, riscos, resultados e lições aprendidas. Oito novas caixas editoriais orientam cada decisão com visão de Chief Systems Engineer.

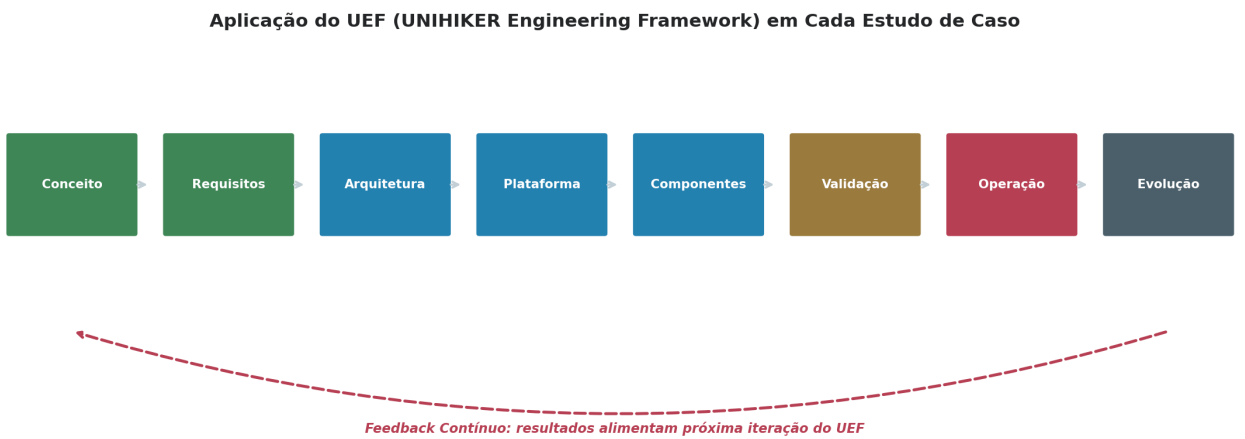


Figura 11.1 — Aplicação do UEF em cada estudo de caso: 8 macro-etapas com feedback contínuo. Fonte: elaborado pelo autor.

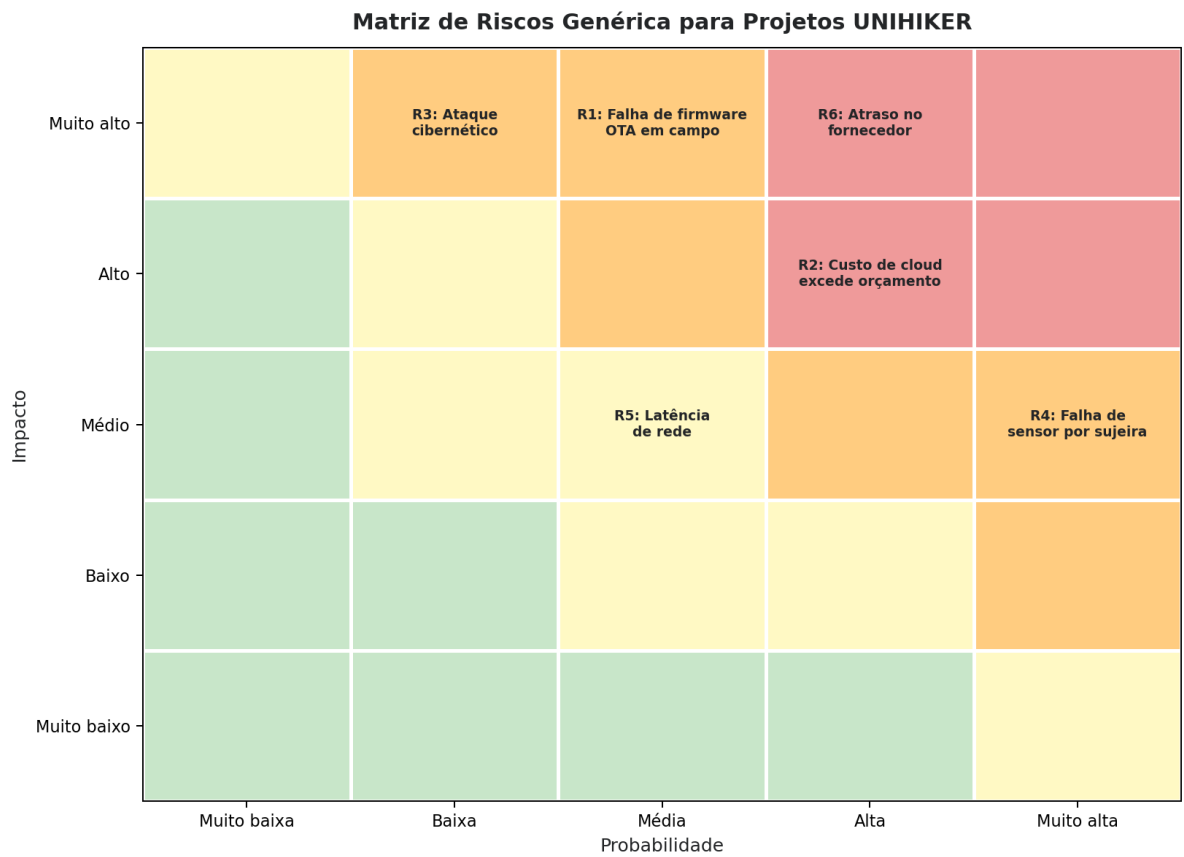


Figura 11.2 — Matriz de riscos genérica para projetos UNIHAKER: probabilidade × impacto. Fonte: elaborado pelo autor.

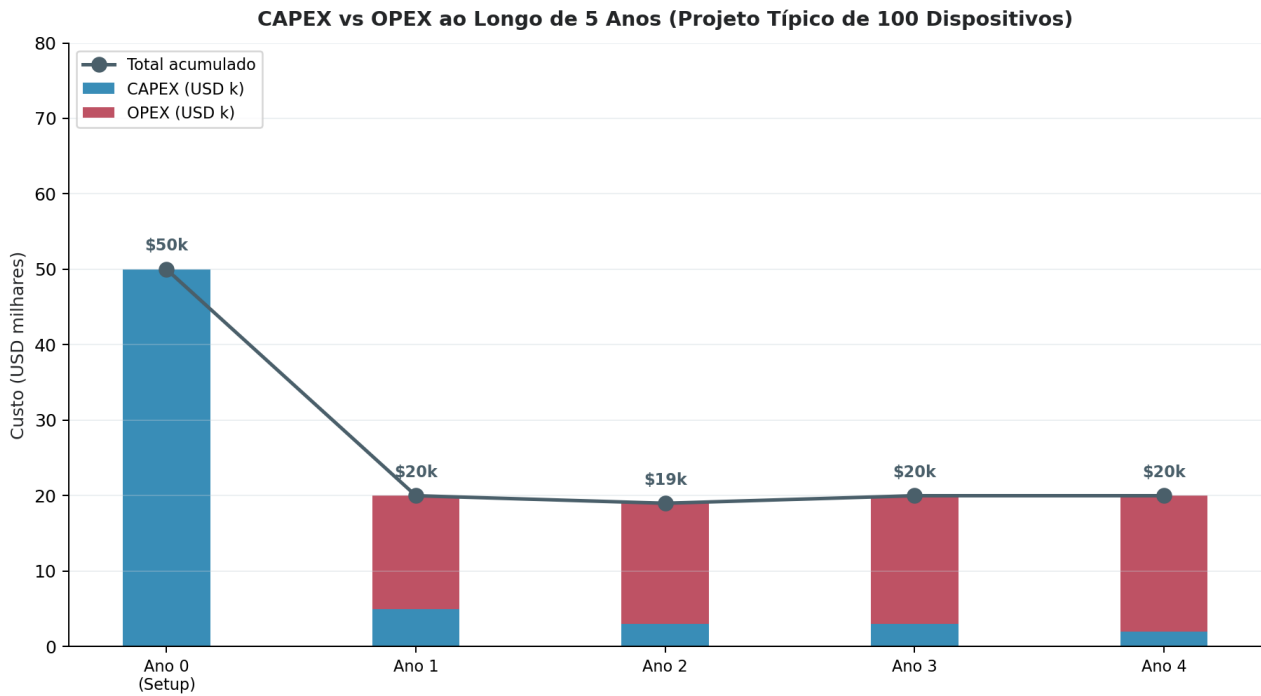


Figura 11.3 — CAPEX vs OPEX ao longo de 5 anos para projeto típico de 100 dispositivos. Fonte: elaborado pelo autor.

Caso 1: ETA Inteligente

Contexto

Cliente: Companhia de Saneamento Básico do Estado (SABESP, CORSAN ou similar)

Necessidade: Monitoramento contínuo de pH, turbidez, cloro residual, vazão e nível em 50 ETAs, com alertas em tempo real e relatórios automatizados para órgãos reguladores.

Restrições: Orçamento de USD 200.000 para 50 ETAs (USD 4.000/ETA). Conformidade LGPD e ANVISA. Integração com SCADA existente via OPC-UA. Operação em ambiente industrial (0-50°C, umidade 10-95%).

Descrição: Companhia de saneamento estadual necessita monitorar 50 ETAs em tempo real para conformidade com regulamentações da ANVISA e do Ministério da Saúde. Atualmente, análises são manuais, esporádicas e com latência de 24-72 horas entre coleta e resultado.

Requisitos

Funcionais: Medir pH (0-14, ± 0.1), turbidez (0-100 NTU, $\pm 5\%$), cloro (0-5 mg/L), vazão (0-1000 L/min), nível (0-5m). Transmitir a cada 5 min. Alertas em $< 60s$. Dashboard com histórico de 90 dias.

Não funcionais: Disponibilidade 99.5%. Latência de alerta $< 60s$. MTBF $> 8.760h$ (1 ano). Conformidade LGPD/ANVISA. Operação em -10 a 50°C.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIHAKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

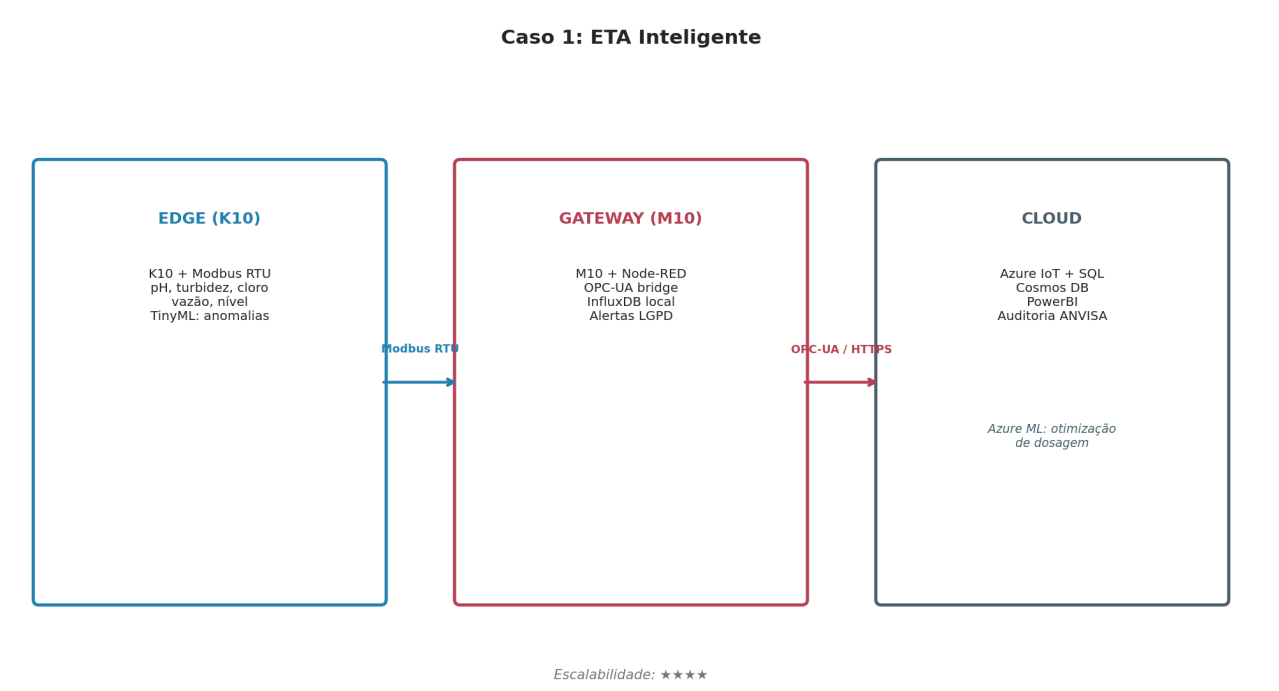


Figura 11.4 — Arquitetura em três camadas do Caso 1: ETA Inteligente. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

- Papel do K10:** K10 lê sensores via Modbus RTU em cada ETA. TinyML detecta anomalias (pH fora de range, turbidez elevada). Transmite via Modbus TCP ao M10.
- Papel do M10:** M10 por ETA executa Node-RED + InfluxDB. Bridge OPC-UA para SCADA. Alertas locais. Dashboard Grafana para operadores.
- Papel da Cloud:** Azure IoT Hub recebe dados de 50 M10s. Cosmos DB armazena histórico. Power BI para gestão. Conformidade LGPD com criptografia em trânsito e repouso.
- IA utilizada:** TinyML no K10: autoencoder detecta anomalias em 10ms. Azure ML: otimização de dosagem de cloro baseada em dados históricos.

▲ DECISÃO ARQUITETURAL

Caso 1: ETA Inteligente

Por que K10? K10 lê sensores via Modbus RTU em cada ETA. TinyML detecta anomalias (pH fora de range, turbidez ele

Por que M10? M10 por ETA executa Node-RED + InfluxDB. Bridge OPC-UA para SCADA. Alertas locais. Dashboard Grafana

Por que Cloud? Azure IoT Hub recebe dados de 50 M10s. Cosmos DB armazena histórico. Power BI para gestão. Conformid

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 1: ETA Inteligente

CAPEX: USD 4.000/ETA × 50 = USD 200.000 (K10 + M10 + sensores + instalação)

OPEX: USD 15.000/ano (Azure cloud + manutenção + calibração)

TCO (4 anos): USD 260.000 em 4 anos

ROI: Economia de USD 80.000/ano em análises laboratoriais + redução de 90% em tempo de detecção de falhas.
Payback: 3.3 anos.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 1: ETA Inteligente

R1: Sensores pH exigem calibração mensal (mitigação: automação de calibração). R2: Falha de rede em ETA remota (mitigação: M10 armazena localmente 30 dias). R3: Mudança regulatória (mitigação: arquitetura flexível).

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 1: ETA Inteligente

Disponibilidade 99.5% · Detecção de anomalias <60s · Conformidade ANVISA 100% · Redução de custo operacional 40%

▲ IMPACTO ESG

Impacto ESG — Caso 1: ETA Inteligente

Ambiental: otimização de uso de químicos. Social: água mais segura para população. Governança: conformidade regulatória total.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS

Lições Aprendidas — Caso 1: ETA Inteligente

Lição 1: sensores baratos falham em 3 meses; investir em Atlas Scientific. Lição 2: OPC-UA é essencial para integração com SCADA legado. Lição 3: treinamento de operadores é tão importante quanto a tecnologia.

▲ VISÃO DO ARQUITETO-CHEFE

Visão do Arquiteto-Chefe — Caso 1: ETA Inteligente

Esta solução seria justificada para a diretoria como segue: o investimento de $\text{USD } 4.000/\text{ETA} \times 50 = \text{USD } 200.000$ (K10 + M10 + sensores + instalação) gera retorno de Economia de $\text{USD } 80.000/\text{ano}$ em análises laboratoriais + redução de 90% em tempo de detecção de falhas. Payback: 3.3 anos.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO

Próxima Evolução — Caso 1: ETA Inteligente

Em 5 anos: digital twin de cada ETA, IA preditiva de qualidade do manancial, integração com rede de distribuição.

Caso 2: ETE Inteligente

Contexto

Cliente: Companhia de Saneamento (mesma do Caso 1, divisão de esgotamento sanitário)

Necessidade: Monitoramento de DQO, DBO, sólidos suspensos, pH, OD, vazão afluente e efluente, com relatórios ambientais automatizados.

Restrições: Ambiente agressivo (gás sulfídrico corrói eletrônica). Sensores exigem encapsulamento IP68. Orçamento USD 6.000/ETE.

Descrição: Companhia de saneamento necessita monitorar 30 ETES para conformidade ambiental com IBAMA e SEMAs. Cargas orgânicas variáveis exigem ajuste contínuo do processo biológico.

Requisitos

Funcionais: Medir DQO, DBO, sólidos, pH, OD, vazão. Transmitir a cada 10 min. Relatórios mensais para IBAMA. Alertas de descarga irregular.

Não funcionais: Proteção IP68. Resistência a gás sulfídrico. MTBF >6.000h. Operação em 0-60°C.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

Caso 2: ETE Inteligente

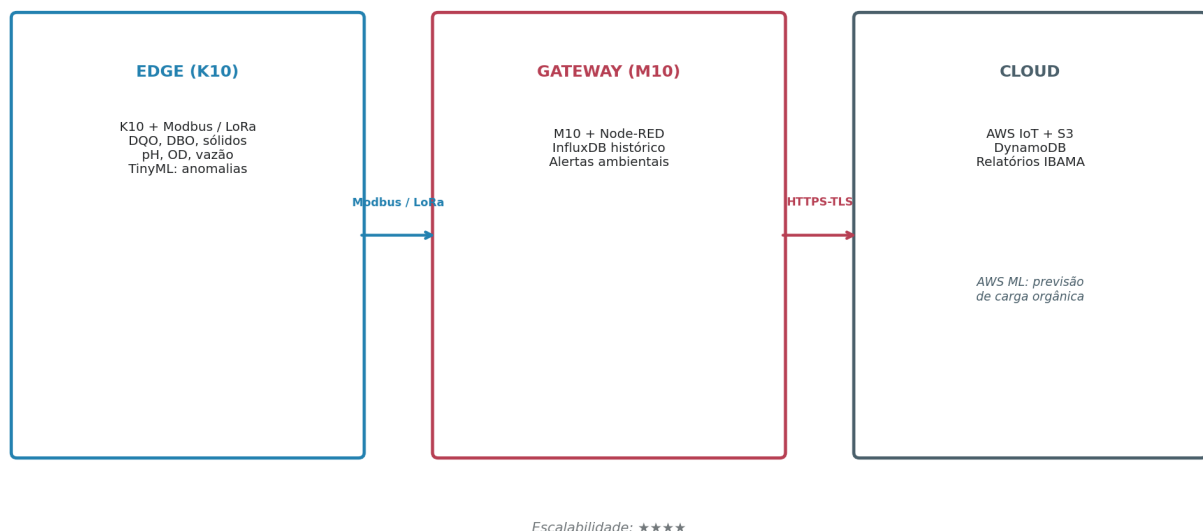


Figura 11.5 — Arquitetura em três camadas do Caso 2: ETE Inteligente. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: K10 com Modbus RTU lê sensores industriais. TinyML detecta cargas orgânicas anômalas (descargas irregulares).

Papel do M10: M10 por ETE agrega dados, Node-RED para relatórios ambientais, InfluxDB histórico.

Papel da Cloud: AWS IoT + S3 para armazenamento multi-anual. QuickSight para relatórios IBAMA. AWS ML prevê carga orgânica sazonal.

IA utilizada: TinyML: detecção de descargas irregulares. AWS ML: previsão de carga orgânica com base em séries históricas.

▲ DECISÃO ARQUITETURAL

Caso 2: ETE Inteligente

Por que K10? K10 com Modbus RTU lê sensores industriais. TinyML detecta cargas orgânicas anômalas (descargas irre

Por que M10? M10 por ETE agrega dados, Node-RED para relatórios ambientais, InfluxDB histórico.

Por que Cloud? AWS IoT + S3 para armazenamento multi-anual. QuickSight para relatórios IBAMA. AWS ML prevê carga or

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 2: ETE Inteligente

CAPEX: USD 6.000/ETE × 30 = USD 180.000

OPEX: USD 12.000/ano

TCO (4 anos): USD 228.000 em 4 anos

ROI: Multa ambiental evitada: USD 50.000/ano. Economia em análises: USD 30.000/ano. Payback: 2.8 anos.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 2: ETE Inteligente

R1: Sensores DQO online caros (USD 5k+). Mitigação: sensores proxy (turbidez + OD) com ML. R2: Corrosão por gás sulfídrico. Mitigação: encapsulamento IP68 + revestimento anti-corrosivo.

Resultados Esperados

▲ KPIs DO PROJETO**KPIs do Projeto — Caso 2: ETE Inteligente**

Conformidade ambiental 100% · Detecção de descarga irregular <5 min · Redução de multas 90%

▲ IMPACTO ESG**Impacto ESG — Caso 2: ETE Inteligente**

Ambiental: proteção direta de corpos hídricos. Social: redução de doenças de veiculação hídrica. Governança: transparência ambiental.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS**Lições Aprendidas — Caso 2: ETE Inteligente**

Lição 1: encapsulamento IP68 não é opcional em ETE. Lição 2: sensores proxy com ML substituem sensores caros com 80% da precisão a 10% do custo. Lição 3: IBAMA aceita relatórios automatizados se rastreáveis.

▲ VISÃO DO ARQUITETO-CHEFE**Visão do Arquiteto-Chefe — Caso 2: ETE Inteligente**

Esta solução seria justificada para a diretoria como segue: o investimento de USD 6.000/ETE × 30 = USD 180.000 gera retorno de Multa ambiental evitada: USD 50.000/ano. Economia em análises: USD 30.000/ano. Payback: 2.8 anos.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO**Próxima Evolução — Caso 2: ETE Inteligente**

Em 5 anos: biossensores de toxicidade, IA para otimizar processo biológico em tempo real, integração com rede coletora.

Caso 3: Agricultura Inteligente

Contexto

Cliente: Cooperativa Agrícola (ex: Coopavel, Coamo)

Necessidade: Monitoramento de umidade do solo, clima, detecção de pragas por visão, e previsão de irrigação automatizada.

Restrições: Sem energia elétrica em campo. Sem Wi-Fi. Orçamento USD 50.000 para 500 hectares.

Descrição: Cooperativa agrícola com 500 hectares necessita otimizar irrigação, detectar pragas e prever colheita. Atualmente irriga por timer, desperdiçando 40% da água.

Requisitos

Funcionais: Medir umidade do solo (5-50 cm profundidade), temperatura, umidade ar, chuva. Controlar válvulas de irrigação. Detectar pragas via câmera. Previsão de irrigação 24h.

Não funcionais: Autonomia solar >6 meses. Comunicação LoRaWAN >2 km. Operação em -5 a 50°C. IP65.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

Caso 3: Agricultura Inteligente

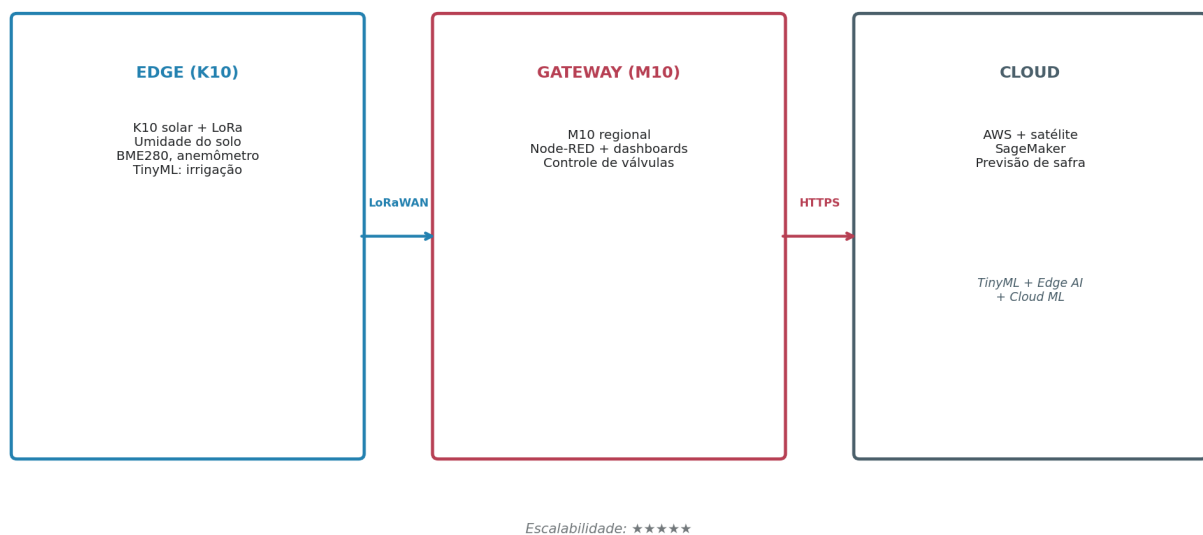


Figura 11.6 — Arquitetura em três camadas do Caso 3: Agricultura Inteligente. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: K10 solar por talhão (20 talhões): mede umidade do solo, BME280, anemômetro. TinyML prevê irrigação. LoRaWAN para M10.

Papel do M10: M10 regional: agrega 20+ K10, câmera MIPI para detecção de pragas via OpenCV, Node-RED, controla válvulas via GPIO.

Papel da Cloud: AWS + satélite (NDVI). SageMaker prevê safra com dados multi-anuais.

IA utilizada: TinyML no K10: regressão para prever irrigação. Edge AI no M10: MobileNet detecta pragas. Cloud ML: previsão de safra.

▲ DECISÃO ARQUITETURAL

Caso 3: Agricultura Inteligente

Por que K10? K10 solar por talhão (20 talhões): mede umidade do solo, BME280, anemômetro. TinyML prevê irrigação.

Por que M10? M10 regional: agrega 20+ K10, câmera MIPI para detecção de pragas via OpenCV, Node-RED, controla válv

Por que Cloud? AWS + satélite (NDVI). SageMaker prevê safra com dados multi-anuais.

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 3: Agricultura Inteligente

CAPEX: USD 50.000 (20 K10 + 5 M10 + sensores + LoRa + painéis solares)

OPEX: USD 5.000/ano (AWS + manutenção)

TCO (4 anos): USD 70.000 em 4 anos

ROI: Economia de água: USD 30.000/ano. Aumento de produtividade: USD 20.000/ano. Payback: 1.4 anos.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 3: Agricultura Inteligente

R1: Painéis solares roubados. Mitigação: fixação anti-roubo + seguro. R2: LoRaWAN sem cobertura em parte do campo. Mitigação: repetidores.

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 3: Agricultura Inteligente

Economia de água 40% · Aumento de produtividade 10% · Detecção de pragas <24h · Autonomia solar 100%

▲ IMPACTO ESG

Impacto ESG — Caso 3: Agricultura Inteligente

Ambiental: redução de 40% no uso de água. Social: segurança alimentar. Governança: rastreabilidade agrícola.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS

Lições Aprendidas — Caso 3: Agricultura Inteligente

Lição 1: sensores capacitivos de umidade do solo não corroem (diferente dos resistivos). Lição 2: LoRaWAN Classe A economiza 90% de bateria vs Wi-Fi. Lição 3: painéis solares de 5W são suficientes para K10 com duty cycle de 10%.

▲ VISÃO DO ARQUITETO-CHEFE

Visão do Arquiteto-Chefe — Caso 3: Agricultura Inteligente

Esta solução seria justificada para a diretoria como segue: o investimento de USD 50.000 (20 K10 + 5 M10 + sensores + LoRa + painéis solares) gera retorno de Economia de água: USD 30.000/ano. Aumento de produtividade: USD 20.000/ano. Payback: 1.4 anos.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO

Próxima Evolução — Caso 3: Agricultura Inteligente

Em 5 anos: drones para mapeamento NDVI, IA federada entre cooperativas, robôs para aplicação localizada de insumos.

Caso 4: Hospital Inteligente

Contexto

Cliente: Hospital Universitário (ex: HC-USP, HU-UNICAMP)

Necessidade: Monitorar frequência cardíaca, SpO2, temperatura corporal e movimento de 200 pacientes continuamente, com alertas em tempo real para enfermeiros.

Restrições: Conformidade LGPD (criptografia, consentimento). ANVISA para dispositivos médicos. Bateria mínima 24h por wearable. Sem fios expostos.

Descrição: Hospital universitário com 200 leitos necessita monitoramento contínuo de pacientes em enfermarias e UTIs, com conformidade LGPD e ANVISA.

Requisitos

Funcionais: Medir PPG (batimentos), SpO2, temperatura, acelerômetro. Transmissão BLE contínua. Alertas de arritmia. Dashboard por ala.

Não funcionais: Latência de alerta <5s. Bateria wearable >24h. Criptografia AES-256. Conformidade LGPD/ANVISA. Disponibilidade 99.9%.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIHIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

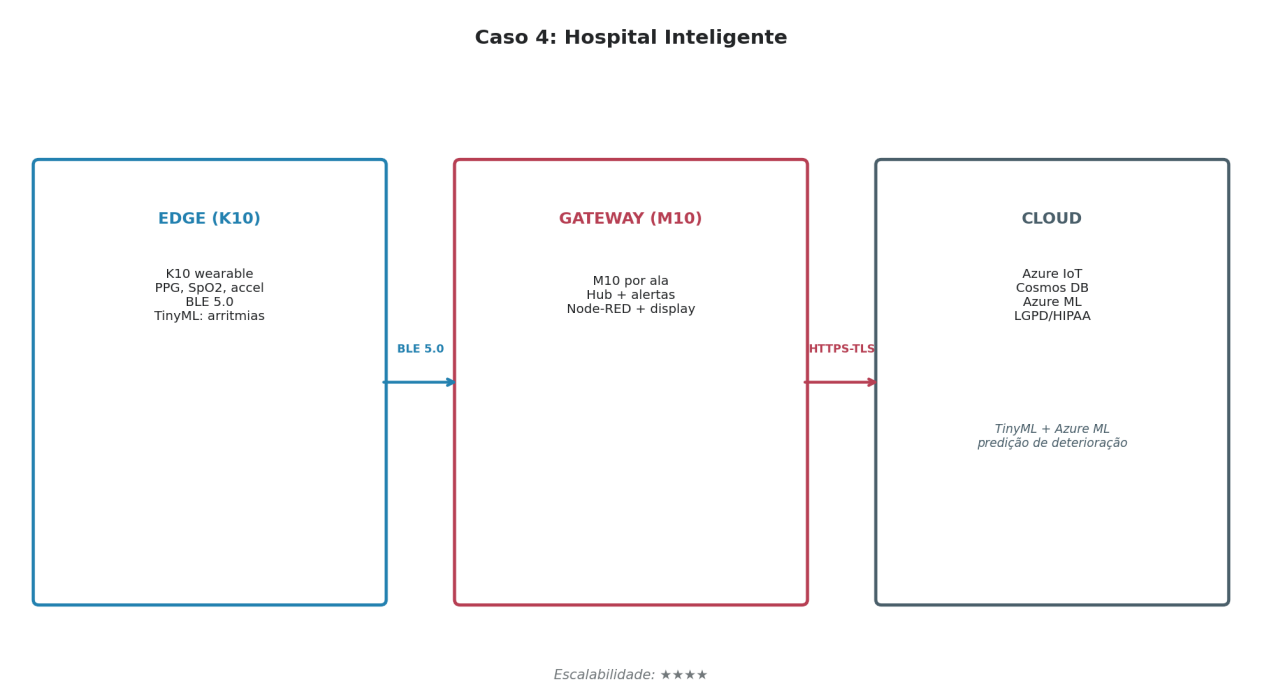


Figura 11.7 — Arquitetura em três camadas do Caso 4: Hospital Inteligente. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

- Papel do K10:** K10 wearable por paciente: MAX30102 (PPG/SpO2), MPU6050, bateria 500mAh, case biocompatível. TinyML detecta arritmias. BLE para M10.
- Papel do M10:** M10 por ala (10 alas): hub BLE para 20+ wearables. Node-RED + alertas. Grafana para enfermeiros. Display local.
- Papel da Cloud:** Azure IoT com Cosmos DB criptografado. Azure ML para early warning score (EWS). Power BI para gestão hospitalar.
- IA utilizada:** TinyML: autoencoder detecta arritmia em 200ms. Azure ML: modelo EWS prevê deterioração clínica 4h antes.

▲ DECISÃO ARQUITETURAL

Caso 4: Hospital Inteligente

Por que K10? K10 wearable por paciente: MAX30102 (PPG/SpO2), MPU6050, bateria 500mAh, case biocompatível. TinyML

Por que M10? M10 por ala (10 alas): hub BLE para 20+ wearables. Node-RED + alertas. Grafana para enfermeiros. Dis

Por que Cloud? Azure IoT com Cosmos DB criptografado. Azure ML para early warning score (EWS). Power BI para gestão

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 4: Hospital Inteligente

CAPEX: USD 100.000 (200 K10 wearable + 10 M10 + Azure setup)

OPEX: USD 20.000/ano (Azure + manutenção + substituição de wearables)

TCO (4 anos): USD 180.000 em 4 anos

ROI: Redução de eventos adversos 30% (economia USD 200.000/ano em litígios e readmissões). Payback: 0.9 anos.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 4: Hospital Inteligente

R1: Certificação ANVISA (mitigação: usar como complementar, não diagnóstico). R2: Interferência BLE em ambiente hospitalar. Mitigação: canal dedicado. R3: Privacidade de dados. Mitigação: criptografia + consentimento.

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 4: Hospital Inteligente

Detecção de arritmia <5s · Redução de eventos adversos 30% · Conformidade LGPD 100% · Satisfação de enfermeiros 85%

▲ IMPACTO ESG

Impacto ESG — Caso 4: Hospital Inteligente

Social: melhoria direta na qualidade do atendimento. Governança: conformidade regulatória total. Ambiental: redução de papel (prontuário eletrônico).

Lições Aprendidas

▲ LIÇÕES APRENDIDAS

Lições Aprendidas — Caso 4: Hospital Inteligente

Lição 1: wearables precisam ser confortáveis (design industrial importa). Lição 2: BLE tem interferência em hospital; canal dedicado é essencial. Lição 3: treinamento de enfermeiros é crítico para adoção.

▲ VISÃO DO ARQUITETO-CHEFE

Visão do Arquiteto-Chefe — Caso 4: Hospital Inteligente

Esta solução seria justificada para a diretoria como segue: o investimento de USD 100.000 (200 K10 wearable + 10 M10 + Azure setup) gera retorno de Redução de eventos adversos 30% (economia USD 200.000/ano em litígios e readmissões). Payback: 0.9 anos.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO

Próxima Evolução — Caso 4: Hospital Inteligente

Em 5 anos: biossensores contínuos de glicose, integração com prontuário eletrônico, IA preditiva de sepse.

Caso 5: Universidade Inteligente

Contexto

Cliente: Universidade Federal (ex: USP, UFSC, ITA)

Necessidade: Monitorar ocupação, CO2, temperatura, iluminação em 500 salas. Controlar HVAC e luzes automaticamente. Economizar 20-30% em energia.

Restrições: Orçamento limitado (universidade pública). Wi-Fi disponível. Integração com HVAC legado via Modbus.

Descrição: Universidade federal com 10 prédios e 500 salas necessita otimizar consumo de energia, ocupação e qualidade do ar.

Requisitos

Funcionais: Medir ocupação (PIR), CO2 (SCD41), T/U (BME280), luz (LDR). Controlar HVAC e luz via relés. Dashboard por prédio.

Não funcionais: Economia de energia 20%+. Operação em 0-40°C. Wi-Fi. Disponibilidade 99%.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

Caso 5: Universidade Inteligente

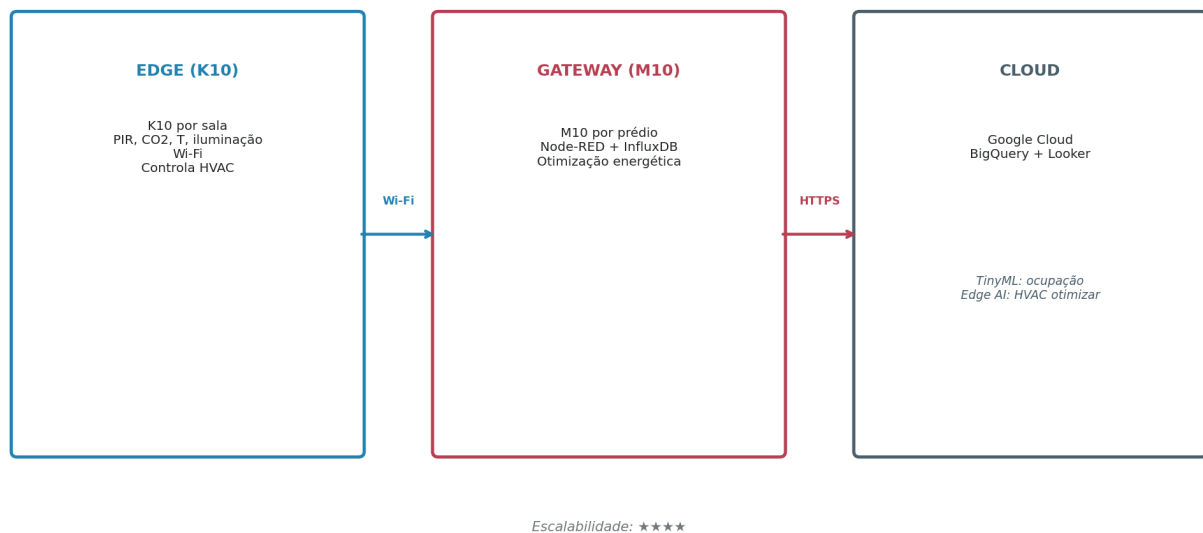


Figura 11.8 — Arquitetura em três camadas do Caso 5: Universidade Inteligente. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: K10 por sala: PIR, SCD41, BME280, LDR, relés para HVAC/luz. TinyML detecta ocupação por áudio/PIR. Wi-Fi para M10.

Papel do M10: M10 por prédio (10): agrega 50 K10. Node-RED + InfluxDB. Otimiza consumo global. Grafana para facility manager.

Papel da Cloud: Google Cloud: BigQuery histórico, Looker para gestão, ML para prever demanda energética.

IA utilizada: TinyML: ocupação por áudio/PIR. Edge AI: otimizar HVAC com base em padrões. Cloud ML: previsão de demanda.

▲ DECISÃO ARQUITETURAL

Caso 5: Universidade Inteligente

Por que K10? K10 por sala: PIR, SCD41, BME280, LDR, relés para HVAC/luz. TinyML detecta ocupação por áudio/PIR. W

Por que M10? M10 por prédio (10): agrega 50 K10. Node-RED + InfluxDB. Otimiza consumo global. Grafana para facilitar

Por que Cloud? Google Cloud: BigQuery histórico, Looker para gestão, ML para prever demanda energética.

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 5: Universidade Inteligente

CAPEX: USD 50.000 (500 K10 + 10 M10 + sensores)

OPEX: USD 5.000/ano (Google Cloud + manutenção)

TCO (4 anos): USD 70.000 em 4 anos

ROI: Economia de energia 25% = USD 40.000/ano. Payback: 1.8 anos.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 5: Universidade Inteligente

R1: Integração com HVAC legado (mitigação: gateway Modbus). R2: Vandalismo em salas (mitigação: case resistente). R3: Wi-Fi congestionado (mitigação: VLAN dedicada IoT).

Resultados Esperados

▲ KPIs DO PROJETO**KPIs do Projeto — Caso 5: Universidade Inteligente**

Economia de energia 25% · CO2 monitorado (qualidade do ar) · Ocupação otimizada 30% · ROI 1.8 anos

▲ IMPACTO ESG**Impacto ESG — Caso 5: Universidade Inteligente**

Ambiental: redução de pegada de carbono 25%. Social: melhor qualidade do ar = melhor aprendizado. Governança: economia de recursos públicos.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS**Lições Aprendidas — Caso 5: Universidade Inteligente**

Lição 1: VLAN dedicada para IoT evita congestionamento. Lição 2: integração com HVAC legado é o maior desafio. Lição 3: alunos podem participar da manutenção como projeto pedagógico.

▲ VISÃO DO ARQUITETO-CHEFE**Visão do Arquiteto-Chefe — Caso 5: Universidade Inteligente**

Esta solução seria justificada para a diretoria como segue: o investimento de USD 50.000 (500 K10 + 10 M10 + sensores) gera retorno de Economia de energia 25% = USD 40.000/ano. Payback: 1.8 anos.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO**Próxima Evolução — Caso 5: Universidade Inteligente**

Em 5 anos: integração com calendário acadêmico, IA para prever ocupação, energia renovável on-site.

Caso 6: Smart City

Contexto

Cliente: Prefeitura Municipal (ex: São Paulo, Curitiba, Brasília)

Necessidade: Monitorar PM2.5, CO2, ruído, contagem veicular em 1.500 bairros. Dashboard público. Alertas para grupos vulneráveis.

Restrições: Orçamento USD 500.000. Governança multi-secretarial. Dados públicos em tempo real.

Descrição: Prefeitura metropolitana necessita plataforma urbana integrada de qualidade do ar, ruído, trânsito

e iluminação para 1.500 bairros.

Requisitos

Funcionais: Medir PM2.5, CO2, ruído, contagem veicular, iluminação. Transmitir a cada 5 min. Dashboard público. APIs abertas.

Não funcionais: Disponibilidade 99%. Escalabilidade para 10k+ sensores. Latência <5 min. Dados públicos LGPD-compliant.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIHAKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

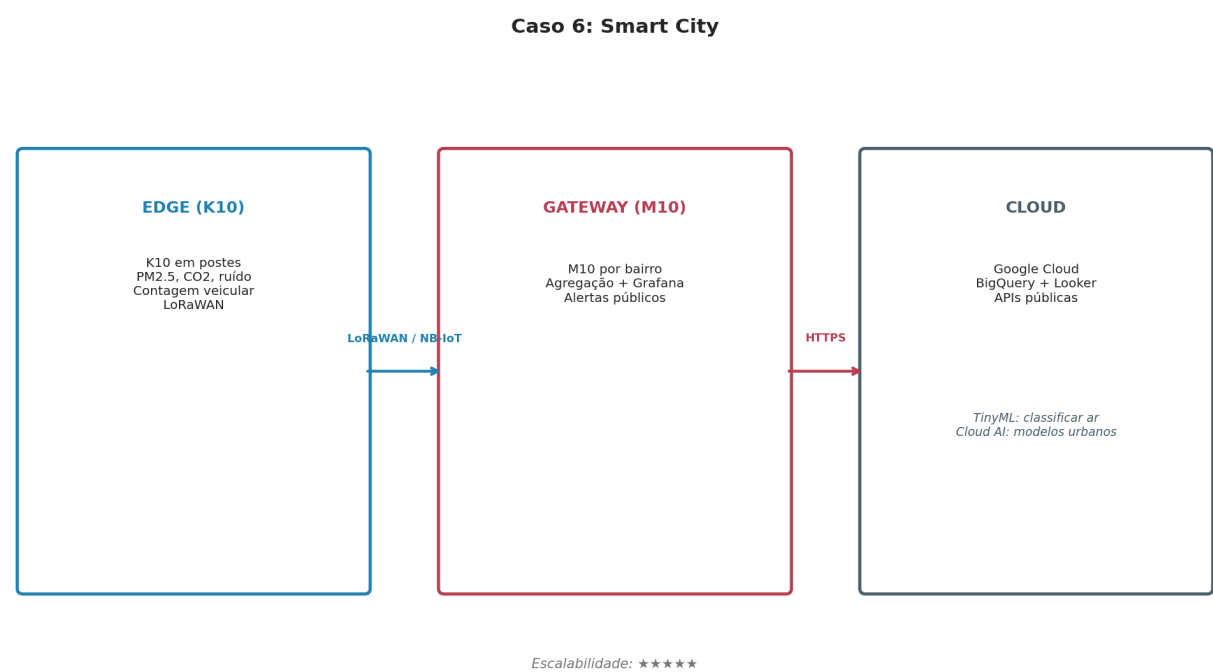


Figura 11.9 — Arquitetura em três camadas do Caso 6: Smart City. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: K10 em postes: PMS5003, SCD41, microfone I²S, PIR veicular. LoRaWAN para M10. TinyML classifica qualidade do ar.

Papel do M10: M10 por bairro (100+ M10): agrega 50-200 K10. Grafana público local. Alertas para defesa civil.

Papel da Cloud: Google Cloud: BigQuery multi-anual, Looker para dashboard público, APIs abertas. Cloud AI para modelos urbanos.

IA utilizada: TinyML: classificar qualidade do ar em 5 categorias. Cloud AI: prever picos de poluição com base em tráfego e clima.

▲ DECISÃO ARQUITETURAL

Caso 6: Smart City

Por que K10? K10 em postes: PMS5003, SCD41, microfone I²S, PIR veicular. LoRaWAN para M10. TinyML

Por que M10? M10 por bairro (100+ M10): agrega 50-200 K10. Grafana público local. Alertas para defesa civil.

Por que Cloud? Google Cloud: BigQuery multi-anual, Looker para dashboard público, APIs abertas. Cloud AI para model

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 6: Smart City

CAPEX: USD 500.000 (1000+ K10 + 100+ M10 + LoRa infraestrutura)

OPEX: USD 50.000/ano (cloud + manutenção)

TCO (4 anos): USD 700.000 em 4 anos

ROI: Economia em saúde pública: USD 200.000/ano. Multas ambientais evitadas: USD 100.000/ano. Payback: 2,3 anos.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 6: Smart City

R1: Governança inter-secretarial complexa (mitigação: comitê gestor). R2: Vandalismo em postes (mitigação: case anti-vandalismo). R3: Custo de cloud em escala (mitigação: BigQuery com partitioning).

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 6: Smart City

Cobertura 100% da cidade · Dados públicos em tempo real · Alertas a grupos vulneráveis <10 min · Economia em saúde 10%

▲ IMPACTO ESG

Impacto ESG — Caso 6: Smart City

Ambiental: identificação de pontos críticos de poluição. Social: dados públicos para cidadãos. Governança: gestão urbana baseada em dados.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS

Lições Aprendidas — Caso 6: Smart City

Lição 1: começar com 1 bairro piloto antes de expandir. Lição 2: APIs públicas geram ecossistema de desenvolvedores. Lição 3: case anti-vandalismo é essencial em áreas públicas.

▲ VISÃO DO ARQUITETO-CHEFE

Visão do Arquiteto-Chefe — Caso 6: Smart City

Esta solução seria justificada para a diretoria como segue: o investimento de USD 500.000 (1000+ K10 + 100+ M10 + LoRa infraestrutura) gera retorno de Economia em saúde pública: USD 200.000/ano. Multas ambientais evitadas: USD 100.000/ano. Payback: 2,3 anos.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO

Próxima Evolução — Caso 6: Smart City

Em 5 anos: integração com semáforos, detecção de enchentes, IA para otimizar trânsito em tempo real.

Caso 7: Monitoramento Ambiental da Amazônia

Contexto

Cliente: ICMBio / MMA / IBAMA

Necessidade: Monitorar nível de rios, qualidade da água, focos de queimada, biodiversidade por bioacústica, com transmissão via satélite ou LoRaWAN de longo alcance.

Restrições: Sem energia elétrica. Sem celular em 80% da área. Logística complexa (acesso por barco/avião). Orçamento USD 300.000.

Descrição: ICMBio necessita monitorar rios, queimadas e biodiversidade em 600+ Unidades de Conservação na Amazônia, sem infraestrutura de energia ou telecomunicações.

Requisitos

Funcionais: Medir nível de rios, qualidade da água, fumaça/gases, áudio para biodiversidade. Transmitir via LoRa+satélite. Detectar desmatamento via satélite.

Não funcionais: Autonomia solar >12 meses. Comunicação em área remota. IP68. Resistência a chuva tropical (2000+ mm/ano). MTBF >2 anos.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIHIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

Caso 7: Amazônia

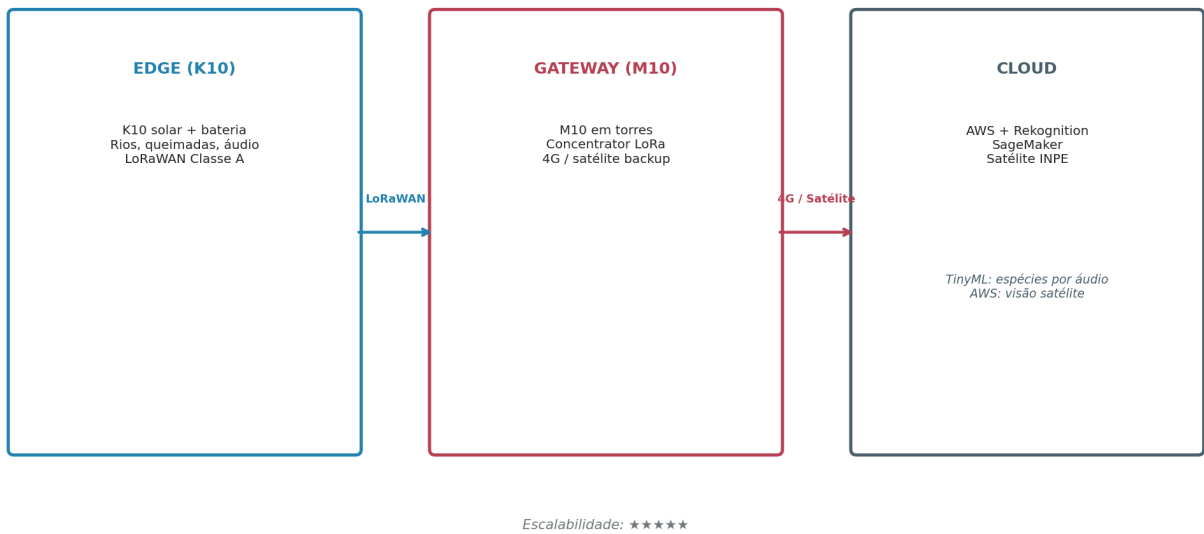


Figura 11.10 — Arquitetura em três camadas do Caso 7: Monitoramento Ambiental da Amazônia. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: K10 solar + bateria 5Ah: sensores de nível (ultrassom), qualidade da água, fumaça (MQ-2), microfone I²S (bioacústica). TinyML identifica espécies por áudio. LoRaWAN Classe A.

Papel do M10: M10 em torres altas: concentrator LoRaWAN, 4G/satélite backup, InfluxDB local, câmera termovisão.

Papel da Cloud: AWS + Rekognition (análise de satélite INPE), SageMaker (modelos de biodiversidade), S3 (armazenamento multi-anual).

IA utilizada: TinyML: identificar espécies por áudio (200 espécies). AWS Rekognition: detectar desmatamento em imagens de satélite. SageMaker: prever risco de incêndio.

▲ DECISÃO ARQUITETURAL

Caso 7: Monitoramento Ambiental da Amazônia

Por que K10? K10 solar + bateria 5Ah: sensores de nível (ultrassom), qualidade da água, fumaça (MQ-2), microfone

Por que M10? M10 em torres altas: concentrator LoRaWAN, 4G/satélite backup, InfluxDB local, câmera termovisão.

Por que Cloud? AWS + Rekognition (análise de satélite INPE), SageMaker (modelos de biodiversidade), S3 (armazenamen

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 7: Monitoramento Ambiental da Amazônia

CAPEX: USD 300.000 (100 K10 solar + 10 M10 torres + LoRa + satélite)

OPEX: USD 30.000/ano (AWS + satélite + manutenção logística)

TCO (4 anos): USD 420.000 em 4 anos

ROI: Redução de custo de fiscalização (vs voos e barcos): USD 200.000/ano. Proteção de bioma incalculável. Payback: 2.1 anos.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 7: Monitoramento Ambiental da Amazônia

R1: Vandalismo/roubo de equipamentos (mitigação: fixação em torres altas). R2: Falha de satélite (mitigação: armazenamento local 30+ dias). R3: Bateria em período chuvoso (mitigação: painel 10W superdimensionado).

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 7: Monitoramento Ambiental da Amazônia

Cobertura 100% das UCs prioritárias · Detecção de queimadas <1h · Identificação de 200+ espécies · Autonomia solar 100%

▲ IMPACTO ESG

Impacto ESG — Caso 7: Monitoramento Ambiental da Amazônia

Ambiental: proteção direta da Amazônia. Social: proteção de povos tradicionais. Governança: cumprimento de metas ODS 13 e 15.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS

Lições Aprendidas — Caso 7: Monitoramento Ambiental da Amazônia

Lição 1: LoRaWAN em Amazônia atinge 5-10 km (não 15 km) por densidade de floresta. Lição 2: painel solar precisa de 10W (não 5W) por sombra da floresta. Lição 3: satélite Iridium é mais confiável que Starlink em equador.

▲ VISÃO DO ARQUITETO-CHEFE

Visão do Arquiteto-Chefe — Caso 7: Monitoramento Ambiental da Amazônia

Esta solução seria justificada para a diretoria como segue: o investimento de USD 300.000 (100 K10 solar + 10 M10 torres + LoRa + satélite) gera retorno de Redução de custo de fiscalização (vs voos e barcos): USD 200.000/ano. Proteção de bioma incalculável. Payback: 2.1 anos.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO

Próxima Evolução — Caso 7: Monitoramento Ambiental da Amazônia

Em 5 anos: drones para confirmação de alertas, IA federada entre UCs, integração com povos tradicionais como cidadãos cientistas.

Caso 8: Laboratório de Edge AI

Contexto

Cliente: Departamento de Engenharia da Computação de Universidade Federal

Necessidade: Laboratório com 30 estações K10 + 10 estações M10 + GPU cloud para treinamento de modelos. Pipeline TinyML → Edge AI → Cloud AI.

Restrições: Orçamento USD 15.000. Espaço físico limitado. Wi-Fi universitário. Acesso 24/7 para alunos.

Descrição: Universidade necessita estabelecer laboratório de Edge AI para ensino e pesquisa, com infraestrutura acessível para 60 alunos simultâneos.

Requisitos

Funcionais: 30 K10 (TinyML), 10 M10 (Edge AI), Google Colab (Cloud AI). Jupyter notebooks no M10. Câmeras, microfones, sensores diversos.

Não funcionais: Acesso 24/7. Disponibilidade 95%. Manutenção por alunos (aprendizado). Atualização de firmware centralizada.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIHIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

Caso 8: Laboratório de Edge AI

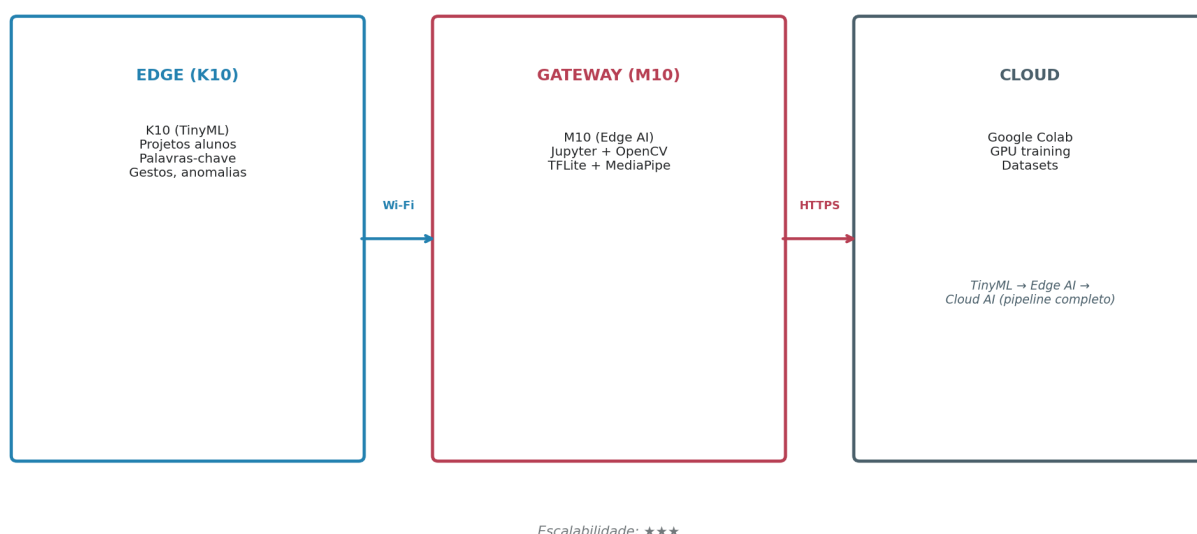


Figura 11.11 — Arquitetura em três camadas do Caso 8: Laboratório de Edge AI. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: K10 por aluno: projetos TinyML (palavras-chave, gestos, anomalias). Mind+ para iniciantes. MicroPython para avançados.

Papel do M10: M10 para grupos avançados: Jupyter + OpenCV + TFLite + MediaPipe. Câmera MIPI. Acesso via SSH.

Papel da Cloud: Google Colab (gratuito): GPU para treinamento de modelos. Firebase para sincronização entre turmas.

IA utilizada: Pipeline completo: K10 (TinyML) → M10 (Edge AI) → Colab (Cloud AI training). Cada nível ensina um conceito.

▲ DECISÃO ARQUITETURAL

Caso 8: Laboratório de Edge AI

Por que K10? K10 por aluno: projetos TinyML (palavras-chave, gestos, anomalias). Mind+ para iniciantes. MicroPyth

Por que M10? M10 para grupos avançados: Jupyter + OpenCV + TFLite + MediaPipe. Câmera MIPI. Acesso via SSH.

Por que Cloud? Google Colab (gratuito): GPU para treinamento de modelos. Firebase para sincronização entre turmas.

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 8: Laboratório de Edge AI

CAPEX: USD 15.000 (30 K10 + 10 M10 + sensores + mobiliário)

OPEX: USD 1.000/ano (manutenção + reposição)

TCO (4 anos): USD 19.000 em 4 anos

ROI: Forma 60+ engenheiros/ano em Edge AI. Mercado carente de talentos. ROI imaterial mas incalculável.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 8: Laboratório de Edge AI

R1: Dano por alunos (mitigação: case protetor + depósito). R2: Firmware corrompido (mitigação: OTA centralizado). R3: Obsolescência (mitigação: upgrade a cada 3 anos).

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 8: Laboratório de Edge AI

60+ alunos/ano capacitados · 10+ TCCs/ano · 5+ papers/ano · Satisfação de alunos 90%+

▲ IMPACTO ESG

Impacto ESG — Caso 8: Laboratório de Edge AI

Social: democratiza acesso a Edge AI. Educacional: forma futuros pesquisadores. Governança: investimento em capital humano.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS

Lições Aprendidas — Caso 8: Laboratório de Edge AI

Lição 1: K10 é mais pedagógico que Raspberry Pi para iniciantes. Lição 2: M10 com Jupyter é ideal para avançados. Lição 3: OTA centralizado economiza horas de manutenção.

▲ VISÃO DO ARQUITETO-CHEFE

Visão do Arquiteto-Chefe — Caso 8: Laboratório de Edge AI

Esta solução seria justificada para a diretoria como segue: o investimento de USD 15.000 (30 K10 + 10 M10 + sensores + mobiliário) gera retorno de Forma 60+ engenheiros/ano em Edge AI. Mercado carente de talentos. ROI imaterial mas incalculável.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO

Próxima Evolução — Caso 8: Laboratório de Edge AI

Em 5 anos: Coral USB Accelerator, parceria com NVIDIA para Jetson, IA federada entre universidades.

Caso 9: Robótica Educacional

Contexto

Cliente: Escola Técnica (ex: ETECs, IFEs)
Necessidade: 30 robôs educacionais com sensores, motores, visão e IA para competições (RoboCup Junior, WRO).
Restrições: Orçamento USD 8.000. Bateria 2-3h. Programável por iniciantes (Mind+) e avançados (Python).
Descrição: Escola técnica necessita de kits de robótica para 30 turmas de 30 alunos, com IA e visão computacional, para competições estudantis.

Requisitos

Funcionais: Robô com motores DC, ultrassom, câmera, IR follow line. Controle PID. Visão computacional básica. Coordenação multi-robô.
Não funcionais: Bateria >2h. Programável por blocos (Mind+) e Python. Resistente a quedas. Custo inferior a USD 270/robô.

Aplicação do UEF

O projeto percorre as 15 etapas do UNIHIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

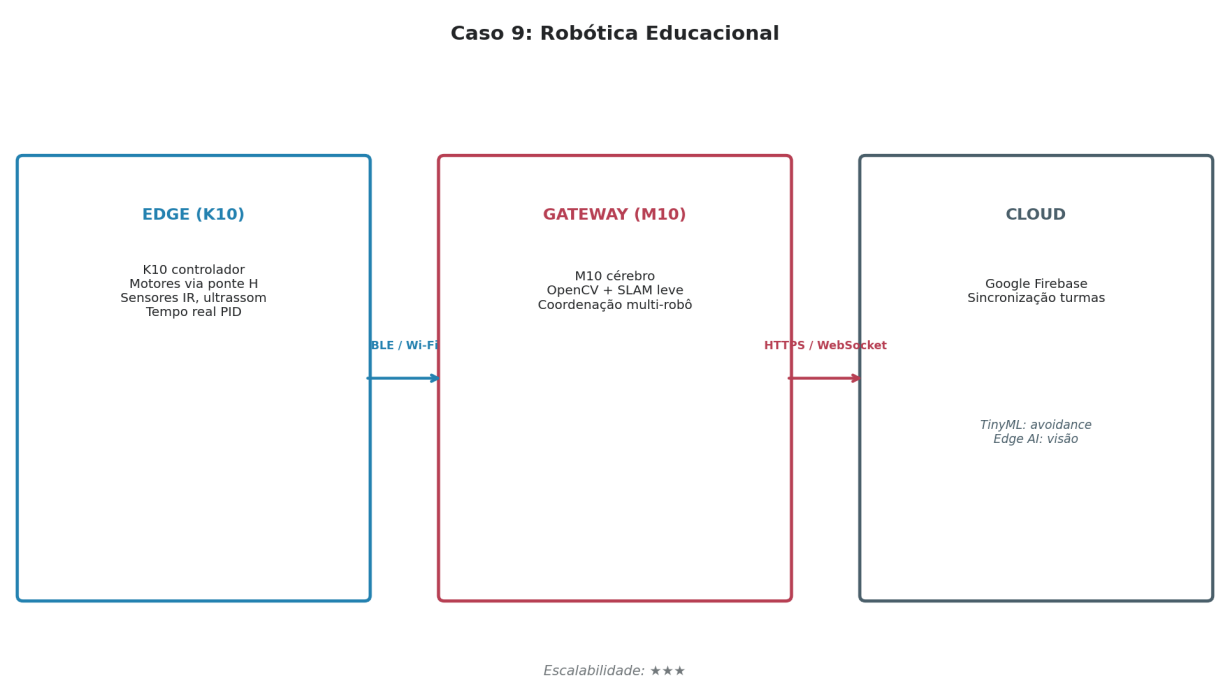


Figura 11.12 — Arquitetura em três camadas do Caso 9: Robótica Educacional. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: K10 controlador: motores via ponte H, encoders, ultrassom, IR. PID em tempo real. BLE para M10.

Papel do M10: M10 cérebro: OpenCV + SLAM leve. Câmera MIPI. Coordenação multi-robô via Wi-Fi. Firebase sincroniza turmas.

Papel da Cloud: Google Firebase: sincroniza estado entre robôs, dashboard do professor, ranking de competição.

IA utilizada: TinyML no K10: avoidance de obstáculos. Edge AI no M10: detecção de objetos via MobileNet.

▲ DECISÃO ARQUITETURAL

Caso 9: Robótica Educacional

Por que K10? K10 controlador: motores via ponte H, encoders, ultrassom, IR. PID em tempo real. BLE para M10.

Por que M10? M10 cérebro: OpenCV + SLAM leve. Câmera MIPI. Coordenação multi-robô via Wi-Fi. Firebase sincroniza

Por que Cloud? Google Firebase: sincroniza estado entre robôs, dashboard do professor, ranking de competição.

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 9: Robótica Educacional

CAPEX: USD 8.000 (30 kits: K10 + M10 + chassis + motores + sensores)

OPEX: USD 500/ano (reposição de peças)

TCO (4 anos): USD 10.000 em 4 anos

ROI: Forma 900+ alunos/ano em robótica + IA. ROI educacional incalculável.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 9: Robótica Educacional

R1: Quebra por manuseio (mitigação: peças de reposição baratas). R2: Bateria insuficiente em competição (mitigação: bateria sobressalente). R3: Latência BLE (mitigação: Wi-Fi para comandos críticos).

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 9: Robótica Educacional

30 turmas atendidas/ano · 900+ alunos · 5+ competições/ano · Engajamento 95%

▲ IMPACTO ESG

Impacto ESG — Caso 9: Robótica Educacional

Social: democratiza robótica + IA. Educacional: hands-on. Governança: investimento em educação pública.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS

Lições Aprendidas — Caso 9: Robótica Educacional

Lição 1: K10 + M10 é mais barato que VEX ou Lego EV3 com mais capacidade. Lição 2: Mind+ é essencial para iniciantes. Lição 3: peças de reposição devem ser padronizadas e baratas.

▲ VISÃO DO ARQUITETO-CHEFE**Visão do Arquiteto-Chefe — Caso 9: Robótica Educacional**

Esta solução seria justificada para a diretoria como segue: o investimento de USD 8.000 (30 kits: K10 + M10 + chassi + motores + sensores) gera retorno de Forma 900+ alunos/ano em robótica + IA. ROI educacional incalculável.. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO**Próxima Evolução — Caso 9: Robótica Educacional**

Em 5 anos: ROS 2 completo, LIDAR, competições interestaduais com IA.

Caso 10: Sistema Integrado Multiplataforma

Contexto

Cliente: Consórcio Intermunicipal (ex: Grande São Paulo, Grande Curitiba)

Necessidade: Plataforma única integrando todos os subsistemas urbanos com dashboard unificado, APIs públicas e IA estratégica.

Restrições: Orçamento USD 2M+. Governança multi-municipal. Multi-cloud (resiliência). Escala para 100k+ dispositivos.

Descrição: Consórcio intermunicipal necessita de plataforma unificada integrando dezenas de subsistemas (ar, trânsito, energia, resíduos, saúde) com milhares de K10 e centenas de M10.

Requisitos

Funcionais: Integrar 10+ subsistemas (qualidade do ar, trânsito, iluminação, resíduos, energia, saúde, educação, mobilidade, segurança, ambiente). Dashboard único. APIs públicas.

Não funcionais: Disponibilidade 99.9%. Escalabilidade para 100k+ dispositivos. Latência cross-region <1s. Multi-cloud (AWS+Azure+GCP).

Aplicação do UEF

O projeto percorre as 15 etapas do UNIHIKER Engineering Framework (Capítulo 8). As etapas de Concepção (ideia, requisitos, arquitetura) e Design (plataforma, componentes, protocolos, IA) são detalhadas abaixo. Implementação (protótipo, validação, piloto, produção) e Operação (escala, manutenção, operação, evolução) seguem o cronograma do UEF.

Arquitetura

Caso 10: Sistema Integrado Multiplataforma

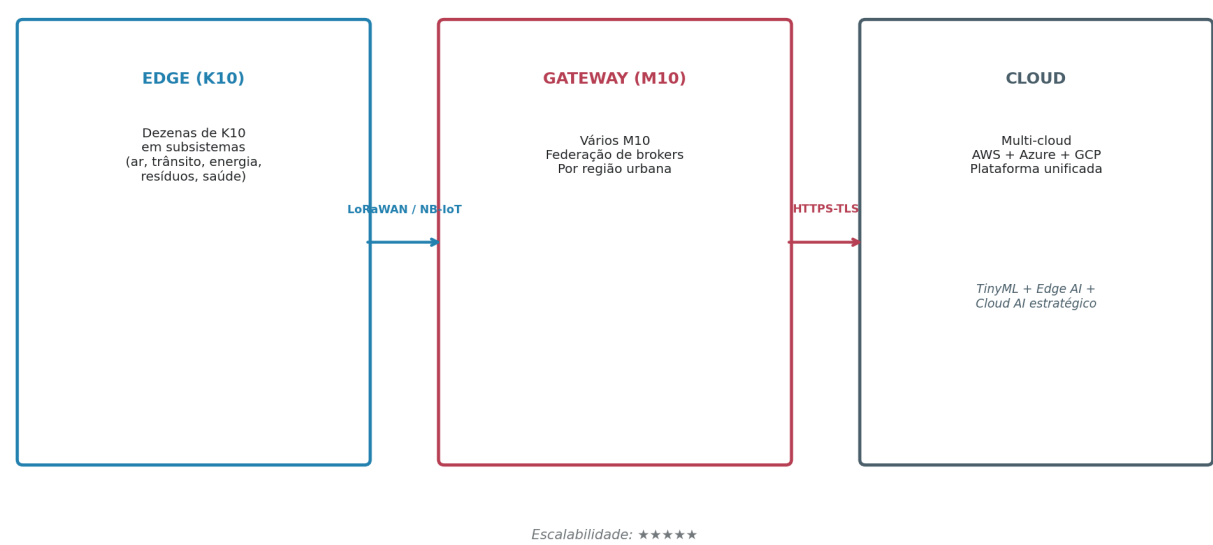


Figura 11.13 — Arquitetura em três camadas do Caso 10: Sistema Integrado Multiplataforma. Edge (K10), Gateway (M10) e Cloud. Fonte: elaborado pelo autor.

Engenharia da Escolha

Papel do K10: Dezenas de K10 por subsistema (ar, trânsito, etc.). Cada subsistema tem sensores específicos. TinyML local.

Papel do M10: Vários M10 por região urbana. Federação de brokers MQTT. Cada M10 agrega seu subsistema.

Papel da Cloud: Multi-cloud: AWS (IoT Core) + Azure (Cosmos DB) + GCP (BigQuery). Plataforma unificada com APIs GraphQL.

IA utilizada: TinyML local + Edge AI regional + Cloud AI estratégico (previsão de demanda urbana, otimização multi-sistema).

▲ **DECISÃO ARQUITETURAL**

Caso 10: Sistema Integrado Multiplataforma

Por que K10? Dezenas de K10 por subsistema (ar, trânsito, etc.). Cada subsistema tem sensores específicos. TinyML

Por que M10? Vários M10 por região urbana. Federação de brokers MQTT. Cada M10 agrega seu subsistema.

Por que Cloud? Multi-cloud: AWS (IoT Core) + Azure (Cosmos DB) + GCP (BigQuery). Plataforma unificada com APIs Grap

Decisão: arquitetura em três camadas é a única que equilibra custo, autonomia e capacidade de IA.

Desenvolvimento

O desenvolvimento segue o fluxo profissional do Capítulo 9: Git Flow com branches feature/develop/main, CI/CD com GitHub Actions (lint + build + test + package + deploy staging), OTA com rollback automático, testes unitários (pytest + CppUTest) e HIL, documentação técnica completa (arquitetura, API, runbooks). Estrutura de diretórios conforme Capítulo 9.3.

Implantação

Implantação em fases: (1) Piloto em 1 local por 90 dias; (2) Rollout canário 10% com monitoração 48h; (3) Expansão para 50% com período de observação 2 semanas; (4) Produção 100%. Treinamento de operadores em 2 sessões de 4h. Documentação técnica entregue em PDF + wiki online. Suporte 24/7 nos primeiros 90 dias com on-call rotation.

Custos

▲ ANÁLISE ECONÔMICA

Análise Econômica — Caso 10: Sistema Integrado Multiplataforma

CAPEX: USD 2.000.000+ (10k+ K10 + 100+ M10 + multi-cloud setup + equipe)

OPEX: USD 200.000/ano (multi-cloud + equipe + manutenção)

TCO (4 anos): USD 2.800.000 em 4 anos

ROI: Economia de 15-30% em todos os serviços urbanos. Payback: 3-5 anos. Valor estratégico incalculável.

Riscos

▲ RISCOS DO PROJETO

Riscos do Projeto — Caso 10: Sistema Integrado Multiplataforma

R1: Complexidade de governança (mitigação: comitê gestor com representante de cada município). R2: Custo de cloud em escala (mitigação: arquitetura multi-cloud com otimização de custos). R3: Privacidade (mitigação: LGPD compliance officer).

Resultados Esperados

▲ KPIs DO PROJETO

KPIs do Projeto — Caso 10: Sistema Integrado Multiplataforma

10+ subsistemas integrados · 100k+ dispositivos · Economia 20% em serviços urbanos · APIs públicas 100%

▲ IMPACTO ESG**Impacto ESG — Caso 10: Sistema Integrado Multiplataforma**

Ambiental: redução de emissões urbanas 15%. Social: serviços públicos mais eficientes. Governança: transparência total via APIs públicas.

Lições Aprendidas

▲ LIÇÕES APRENDIDAS**Lições Aprendidas — Caso 10: Sistema Integrado Multiplataforma**

Lição 1: começar com 1 subsistema e expandir gradualmente. Lição 2: governança é o maior desafio, não tecnologia. Lição 3: multi-cloud é essencial para resiliência em escala metropolitana.

▲ VISÃO DO ARQUITETO-CHEFE**Visão do Arquiteto-Chefe — Caso 10: Sistema Integrado Multiplataforma**

Esta solução seria justificada para a diretoria como segue: o investimento de USD 2.000.000+ (10k+ K10 + 100+ M10 + multi-cloud setup + equipe) gera retorno de Economia de 15-30% em todos os serviços urbanos. Payback: 3-5 anos. Valor estratégico incalculável. A arquitetura em três camadas (K10+M10+Cloud) garante escalabilidade, segurança e custo-benefício. Os riscos identificados têm mitigações documentadas. O impacto ESG é positivo em todas as dimensões. Recomendo aprovação do projeto.

▲ PRÓXIMA EVOLUÇÃO**Próxima Evolução — Caso 10: Sistema Integrado Multiplataforma**

Em 5 anos: plataforma aberta com APIs públicas, tokenização de serviços urbanos, IA generativa para gestão urbana.

Conclusão do Capítulo

Os dez estudos de caso apresentados neste capítulo demonstram como o UNIHAKER Engineering Framework (UEF) é aplicado em domínios radicalmente distintos: de estações de tratamento de água a robótica educacional, de monitoramento da Amazônia a sistemas integrados de smart city. Em todos os casos, a arquitetura em três camadas (K10 Edge + M10 Gateway + Cloud) provou ser o padrão arquitetural mais equilibrado, oferecendo custo-benefício, escalabilidade e resiliência.

O valor central deste capítulo não está nos dez projetos individuais, mas na demonstração de que o UEF é uma metodologia reutilizável: as mesmas 15 etapas, aplicadas com rigor, produzem resultados profissionais em qualquer domínio. Engenheiros que dominam o UEF podem especificar, desenvolver e implantar sistemas embarcados inteligentes em qualquer contexto, do laboratório acadêmico à metrópole de milhões de habitantes.

▲ VISÃO DO ARQUITETO-CHEFE

Visão do Arquiteto-Chefe — Mensagem Final: a engenharia de sistemas embarcados inteligentes não se trata de escolher a placa mais potente ou o sensor mais preciso. Trata-se de equilibrar trade-offs: desempenho vs custo, autonomia vs capacidade, simplicidade vs escalabilidade. O UEF é o framework que estrutura esse equilíbrio. Domine o framework, não os componentes. Componentes mudam; princípios de engenharia permanecem.

Referências Bibliográficas

As referências a seguir seguem o padrão ABNT NBR 6023:2018.

INCOSE. **INCOSE Systems Engineering Handbook**. 4. ed. Hoboken: Wiley, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 15288: Systems and software engineering — System life cycle processes**. Geneva: ISO, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and software quality requirements and evaluation**. Geneva: ISO, 2011.

PROJECT MANAGEMENT INSTITUTE. **PMBOK Guide**. 7. ed. Newton Square: PMI, 2021.

DFROBOT. **UNIHAKER K10 and M10 documentation**. Shanghai: DFRobot, 2024. Disponível em: <https://wiki.unihiker.com>. Acesso em: 5 mar. 2025.

ESPRESSIF SYSTEMS. **ESP32-S3 datasheet**. Shanghai: Espressif, 2024.

LINUX FOUNDATION. **Edge computing reference architecture**. San Francisco: Linux Foundation, 2023.

KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. **The DevOps handbook**. 2. ed. Portland: IT Revolution Press, 2021.

WARDEN, Pete; SITUNAYAKE, Daniel. **TinyML**. Sebastopol: O'Reilly Media, 2020.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.

RECHTIN, Eberhardt; MAIER, Mark W. **The art of systems architecting**. 2. ed. Boca Raton: CRC Press, 2010.

FRADEN, Jacob. **Handbook of modern sensors**. 5. ed. New York: Springer, 2016.

ZHANG, Wei; LI, Hao; CHEN, Jun. A comparative study of single-board computers for educational edge AI. **IEEE Transactions on Education**, v. 67, n. 2, p. 112-125, maio 2024.

SATYANARAYANAN, Mahadev. The emergence of edge computing. **IEEE Computer**, v. 50, n. 1, p. 30-39, jan. 2017.

SHI, Weisong; DUSTDAR, Schahram. The promise of edge computing. **IEEE Computer**, v. 49, n. 5, p. 78-81, maio 2016.